

2026 상반기 DFIR 기초 역량 강화 세미나

Android

2026-03-13

김은채, 조민혁



고려대학교 정보보호대학원
Korea University
School of Cybersecurity



Korea University
Digital Forensic Research Center

- 개요
- 데이터 수집 방법
- 데이터 분석 방법
- 도구 실습

개요

Android 정의

■ 모바일 운영체제 개념

- 스마트폰, 태블릿, 자동차, 웨어러블 기기 등 자원 제약 환경에서 탑재되는 운영체제
- 데스크탑 운영체제와 비슷하지만 설계 측면에서 차이 존재
- 노트북도 모바일 기기지만 데스크탑 운영체제가 탑재되어 기능적으로 모바일 기기와 차이 존재

구분	특징	모바일 운영체제	데스크탑 운영체제
공통점	커널 기반 구조	프로세스 스케줄링, 가상 메모리 관리, 파일 시스템, 네트워크, 장치 드라이버 관리 등 필요	
	보안 메커니즘	권한 기반 접근 제어, 사용자 데이터 보호	
	애플리케이션 실행	API 및 라이브러리 제공	
차이점	하드웨어 구조	저전력 기반 SoC (ARM)	고성능 CPU (x86 / 고성능 ARM)
	설계 우선 순위	배터리 효율, 최소 자원 사용	성능 최우선
	소프트웨어 배포	중앙 집중형	자유로운 설치
	파일시스템	앱 별 격리된 저장 공간	모바일 OS에 비해 상대적으로 개방형 구조

Android 정의

■ 모바일 운영체제 개념

- 모바일 운영체제 시장은 Android와 iOS 중심 구조
- Android는 AOSP (Android Open Source Project)로 코드 공개, Apache License 2.0 사용
 - AOSP: 구글이 주도해 개발하는 오픈소스 운영체제 프로젝트
 - Apache License 2.0: 상업적 사용 가능, 소스코드 공개 의무 X, 수정 후 독점 배포 가능

운영체제	개발사	계열	프로그래밍	특징	점유율
Android 	Google, OHA	Linux 기반	C, C++, Java, Kotlin, Rust	오픈 소스 (AOSP) 커스터마이징 가능 → 다양성	68.24%
iOS HarmonyOS	Apple Inc.	Unix 계열 Darwin	C, C++, Swift, Objective - C	폐쇄적 생태계, 강력한 보안 통제 하드웨어 소프트웨어 통합 최적화	31.48%
TIZEN 	Huawei	자체 설계	C, C++, JS, ArkTS	중국 내 집중 사용, 분산 OS 구조	-
	Samsung, Intel	Linux 기반	C++, C#, HTML5, JS	TV, IoT 중심	0.22%
KaiOS	KaiOS Technologies Inc.	Linux 기반	C++, HTML5, JS	피쳐폰 및 저사양 기기 대상	0.01%

Android 정의

■ Android 버전 정보 힘들다

- 공식 명칭의 일관성 및 명확성의 이유로 10 버전 이후 부터 숫자 사용

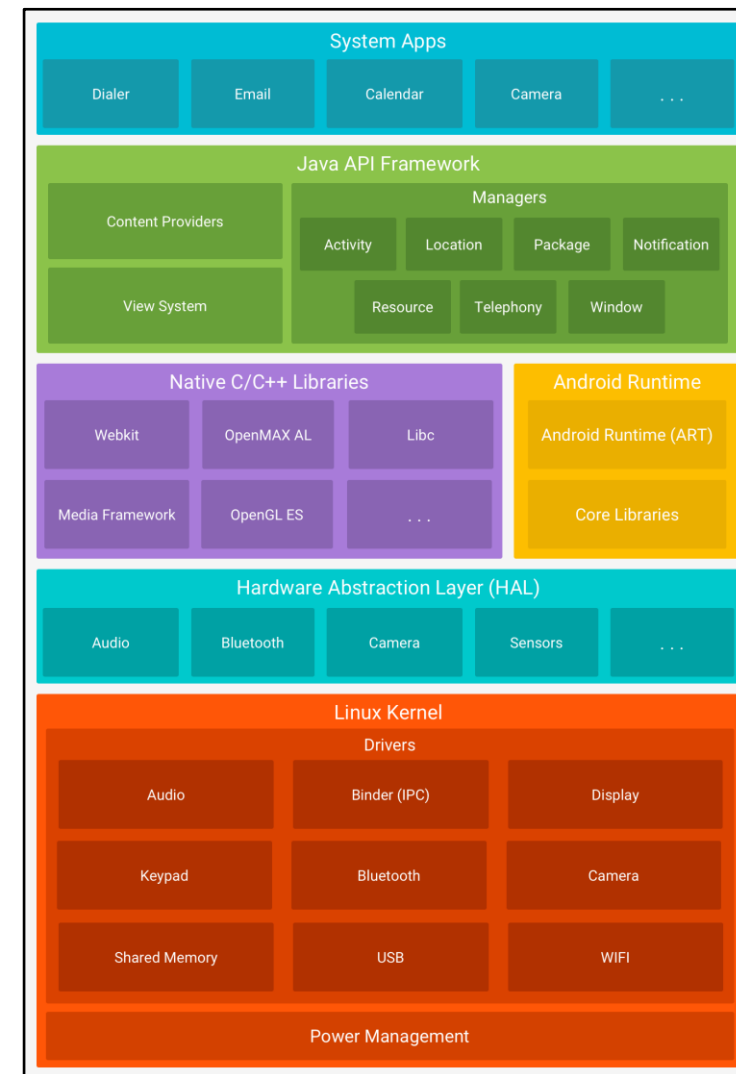
OS 버전	공식 명칭	출시일	주요 특징
1.0	-	2008-09-23	최초 상용 릴리즈
1.1	-	2009-02-09	버그 수정 및 API 개선
1.5	Cupcake	2009-04-30	온스크린 키보드
1.6	Donut	2009-09-15	다양한 화면 해상도 지원
2.0 – 2.1	Eclair	2009-10-26	Google Maps 내비게이션
2.2	Froyo	2010-05-20	JIT 컴파일러 도입
2.3	Gingerbread	2010-12-06	NFC 지원
3.0 – 3.2	Honeycomb	2011-02-22	태블릿 전용
4.0	Ice Cream Sandwich	2011-10-19	얼굴 인식 잠금해제
4.1 – 4.3	Jelly Bean	2012-07-09	Project Butter
4.4	Kitkat	2013-10-31	기기 화면 녹화 지원
5.0	Lollipop	2014-10-16	ART 런타임

OS 버전	공식 명칭	출시일	주요 특징
6.0	Marshmallow	2015-05-28	지문 인증 / 런타임 권한 도입
7.0	Nougat	2016-08-22	멀티 윈도우 지원
8.0	Oreo	2017-08-21	안드로이드 트레블
9.0	Pie	2018-08-06	디스플레이 컷아웃
10.0	Android 10	2019-09-03	다크 모드 지원
11.0	Android 11	2020-09-08	범위 지정 저장소
12.0	Android 12	2021-10-04	프라이버시 대시보드
12.1	Android 12L	2022-03-08	큰 화면을 위한 UI
13.0	Android 13	2022-08-16	앱 권한 세분화
14.0	Android 14	2023-10-04	Target SDK 23 미만 앱 설치 차단
15.0	Android 15	2024-09-03	인앱 카메라 컨트롤
16.0	Android 16	2025-06-03	알림 쿨다운
17.0	Android 17	2026-02-12 (Beta)	

Android 정의

■ Android 아키텍처

- 다양한 기기에 사용될 수 있도록 제작
 - 스마트폰 뿐만 아니라 자동차, 웨어러블, 태블릿 등에 사용 가능
- Linux 기반의 오픈소스 소프트웨어 스택
- 구성요소
 - System Apps
 - Java API Framework
 - Native C / C++ Libraries / Android RunTime (ART)
 - Hardware Abstraction Layer (HAL)
 - Linux Kernel



Android 정의

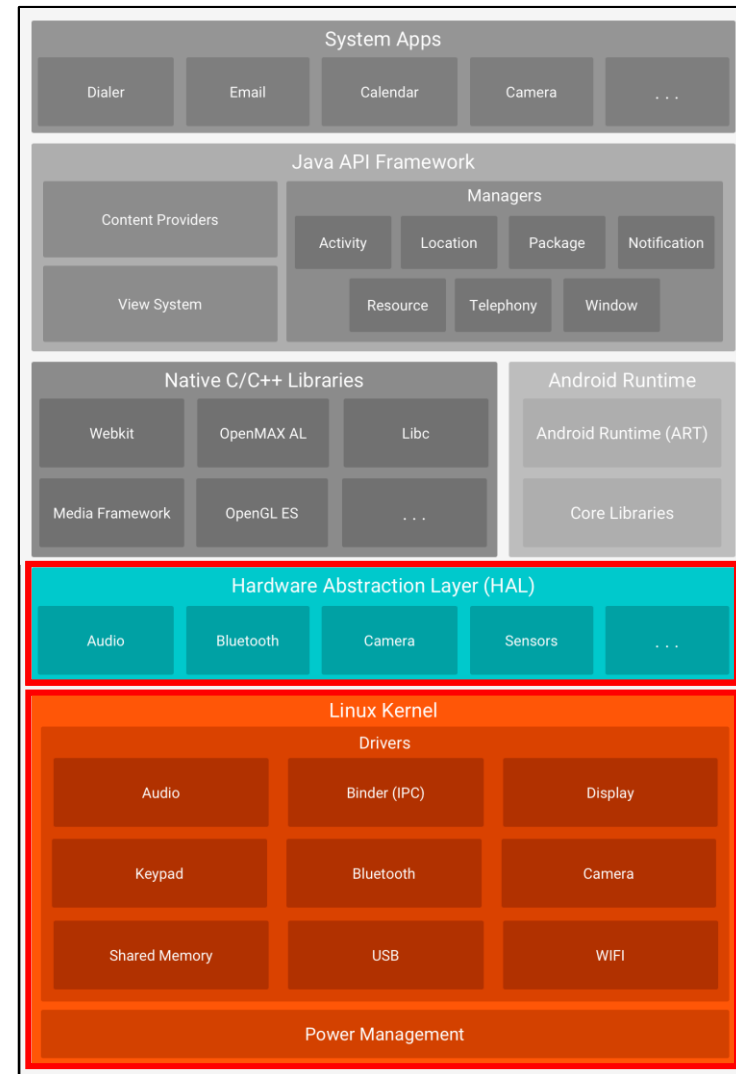
■ Android 아키텍처

■ Linux Kernel

- 하드웨어를 제어하는 최하위 계층
 - 프로세스 관리, 메모리 관리, 디바이스 드라이버 관리 등 수행
- 제조사에서 커널용 하드웨어 드라이버 개발 가능

■ Hardware Abstraction Layer (HAL)

- 하드웨어를 일관된 방식으로 사용할 수 있게 해주는 인터페이스
 - 제조사는 HAL 표준에 맞게 기기용 HAL만 구현
 - Java API Framework는 서로 다른 제조사의 하드웨어를 일관된 방식으로 사용 가능
- 기능별로 여러 라이브러리 모듈 존재 및 커널 드라이버 직접 호출

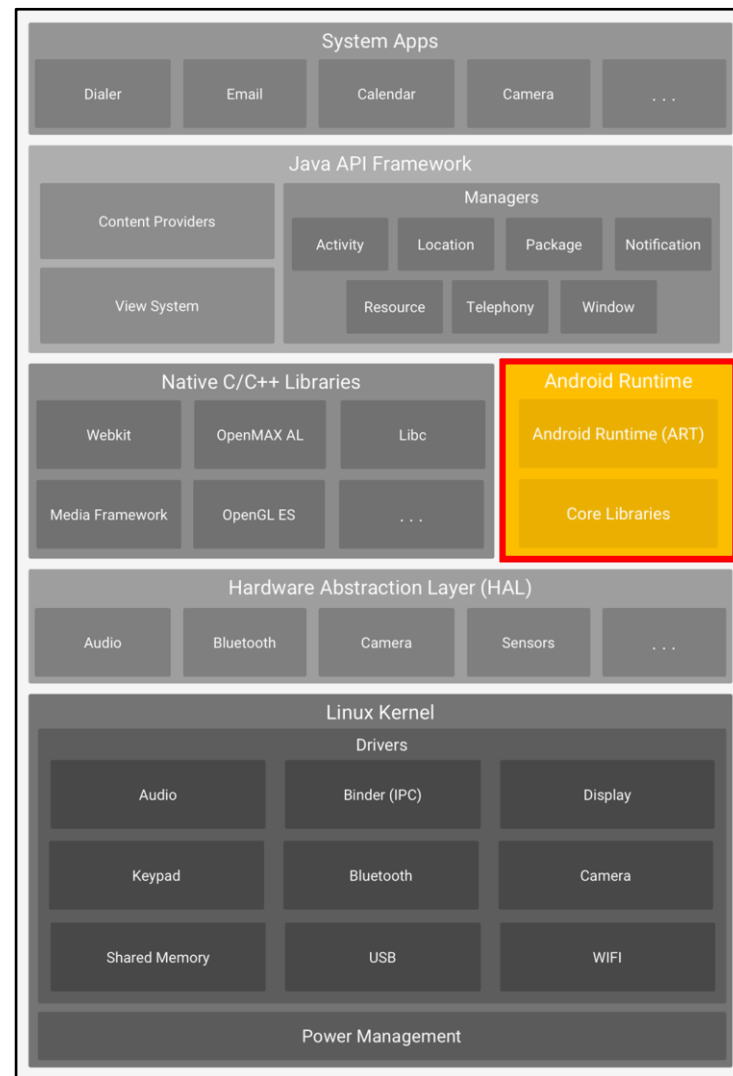


Android 정의

■ Android 아키텍처

■ Android Runtime (ART)

- 앱의 바이트코드를 컴파일하여 실행하는 실행 환경
- DEX 파일을 실행해 동일 앱으로 서로 다른 아키텍처 실행 가능
 - DEX (Dalvik Executable): .class 파일들을 취합한 하나의 파일
- Android 5.0 이전에는 Dalvik VM 사용
 - Dalvik VM: JIT 컴파일 방식을 통해 바이트코드를 실행하는 환경
 - JIT (Just In Time): 자주 사용되는 코드를 미리 컴파일하는 방식
- Dalvik VM의 성능 및 효율 문제로 인해 Android 5.0부터 ART 도입
- 현재 ART는 AOT 및 JIT 컴파일 혼합 사용
 - AOT (Ahead Of Time): 설치 시점에 정적으로 미리 컴파일 수행



Android 정의

■ Android 아키텍처

■ Native C / C++ Libraries

- Android 시스템이 내부적으로 사용하는 네이티브 코드 모음
- 앱은 JNI를 통해 네이티브 라이브러리 사용 가능
 - JNI (Java Native Interface): Java 코드와 네이티브 코드 사이에서 서로 호출 가능하게 하는 인터페이스

■ Java API Framework

- 앱이 시스템 기능을 사용할 수 있도록 제공되는 API
 - 시스템 구성요소 및 서비스 재사용 → 중복 코드 작성 방지

■ System Apps

- 기기에 기본 포함되며 일부는 높은 권한을 가진 앱



Android 정의

■ Android 아키텍처

■ Android 시스템 앱

- 제조사가 기기 제조 시 미리 탑재하여 출시하는 앱
 - 기본 시스템 앱과 특권 시스템 앱 존재
- 대표 시스템 앱: 계산기, 설정, 시스템 UI, 하드웨어 가속 앱 등
- 저장 경로
 - APK: /system/app, /system/priv-app, /product/app, /vendor/app
 - 앱 데이터: /data/data/<package-name> 경로에 설치
- 접근 권한
 - 기본 시스템 앱: Normal, Dangerous, Signature 권한
 - 특권 시스템 앱: Normal, Dangerous, Signature, Privileged 권한

Android 정의

■ Android 아키텍처

■ Android 사용자 앱

- 사용자가 직접 설치한 앱
 - 기본적으로 시스템 파일 직접 접근 불가, 필요시 정책 준수하여 API 요청 필요
- 대표 사용자 앱: 카카오톡, 네이버, 쿠팡 등
- 저장 경로
 - APK: /data/app
 - 앱 데이터: /data/data/<package-name>
- 접근 권한: Normal¹⁾, Dangerous²⁾, Signature³⁾ 권한

1) Normal: 사용자 정보나 시스템에 미치는 영향이 적은 권한/ 사용자가 별도 승인하지 않아도 앱 설치시 자동 허용

2) Dangerous: 사용자 정보나 민감한 시스템 자원 접근 권한

3) Signature: 동일한 인증서로 서명된 앱만 사용할 수 있는 권한 / 시스템 앱 혹은 동일 제조사간 앱 간 기능 공유에 사용

Android 정의

■ Android 부팅 모드

- 부트로더 단계에서 결정되는 실행 모드
 - 키 입력, 부트 플래그, 파티션 상태 등을 기준으로 결정
- Normal Boot Mode
 - 일반적으로 사용자가 기기를 켤 때 모드
 - 시스템 서비스 시작, 서드파티 앱 실행 가능, 백그라운드 서비스 실행 등
- Safe Mode
 - /system, /vendor 파티션 앱만 실행 → 서드파티 앱을 비활성화 상태로 부팅
 - 기기 문제 진단용 → 문제가 시스템인지, 서드파티 앱인지 구분 가능
 - 진입 방법
 - 전원 끄기 버튼 Hold → 안전 모드



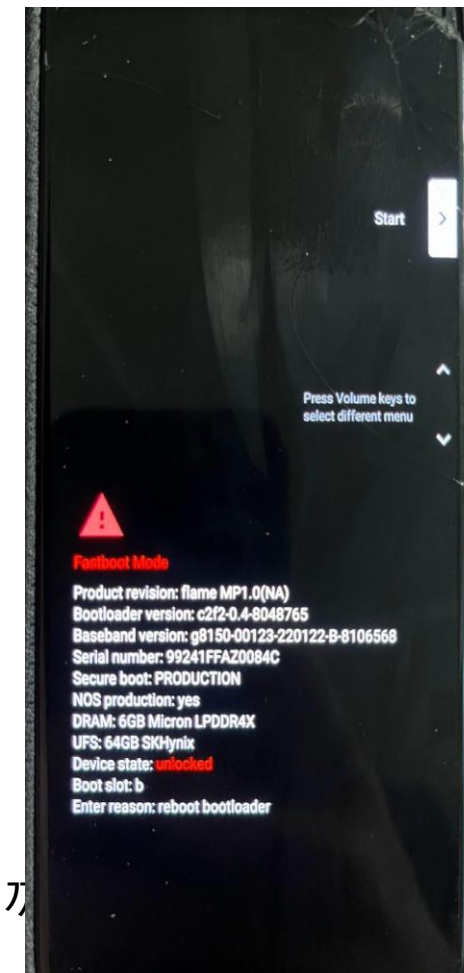
[Safe Mode 진입 모습]

Android 정의

■ Android 부팅 모드

■ Fastboot Mode

- OS를 로드하지 않고 부트로더 단계에서 대기하는 프로토콜 기반 모드
- USB를 통해 PC와 Fastboot 프로토콜로 통신
 - Fastboot 프로토콜: PC와 기기의 부트로더가 명령 및 데이터를 주고받기 위해 사용하는 통신 규약
 - CLI 환경에서 저수준 작업 가능
- 파티션 이미지 플래시, 부트로더 Unlock / Lock 등 가능
- 진입 방법
 - 전원 버튼 + 볼륨 하 (-) 버튼을 동시에 길게 눌러, 재부팅 후 로고가 나올 때까지 기다리기
 - adb reboot bootloader



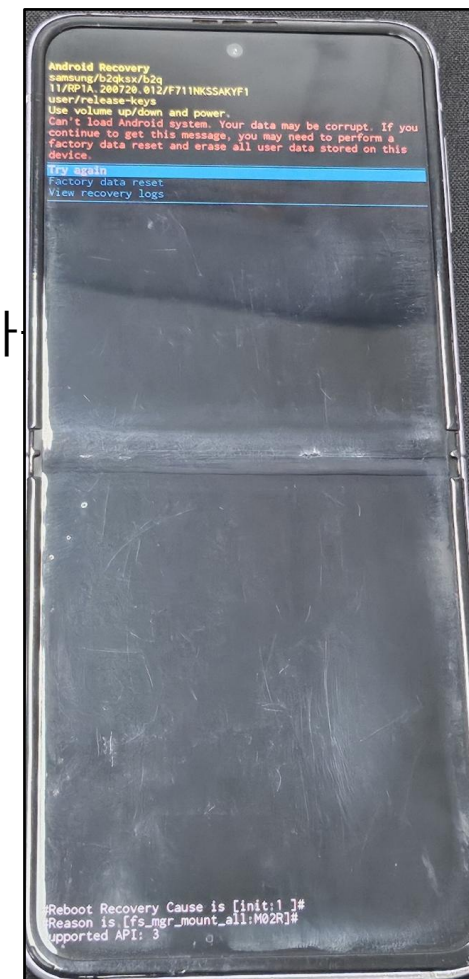
[Fastboot Mode 진입 모습]

Android 정의

■ Android 부팅 모드

■ Recovery Mode

- 기기 문제 해결을 위해 Android OS 대신 복구용 환경을 실행하는 모드
- 별도의 커널과 RamDisk로 구성되어 기존 Android 시스템이 손상되어도 실행 가능
 - RamDisk: Ram을 디스크처럼 사용하는 임시 파일 시스템
- 기기가 부팅 루프에 갇히거나 악성코드에 감염되었을 때 유용
- 주요 기능
 - 시스템 업데이트, 공장 초기화, 캐시 삭제 등
- 진입 방법
 - USB 케이블이 삽입된 상태에서 전원 버튼 + 볼륨 상 (+) 버튼
 - adb reboot recovery



[Recovery Mode 진입 모습]

Android 정의

■ Android 보안 기능

- 스토리지 암호화: 기기 내부 저장소에 저장되는 데이터를 암호화하여 허용되지 않은 사용자가 데이터에 접근하지 못하도록 보호하는 보안 기능
 - 목적
 - 단말이 물리적으로 탈취되었을 경우 내부 데이터 보호
 - 저장소 추출을 통한 허용되지 않은 데이터 접근 금지
 - 동작 과정
 - 사용자 데이터가 저장될 때 디스크에 커밋되기 전에 자동으로 암호화 → 디바이스 스토리지 저장
 - 사용자 인증 정보를 기반으로 암호화 키 관리
 - 인증 성공 시 암호 키가 활성화되어 호출 프로세스로 반환하기 전에 데이터 복호화
- 두 가지의 암호화 방식 존재: FDE (Full Disk Encryption), FBE (File Based Encryption)

Android 정의

■ Android 보안 기능

■ FDE (Full Disk Encryption)

- Android 4.4 ~ 9 버전에서 사용되는 사용자 데이터 영역 전체 암호화 기능
 - Android 4.4: FDE가 옵션으로 처음 도입, Android 5.0: FDE 사용 의무화
- 기기의 단일 고유 키 DEK (Disk Encryption Key)으로 /data 파티션 암호·복호화 수행
- DEK은 사용자 인증 수단과 하드웨어 고유 키를 조합한 KEK (Key Encryption Key)를 통해 암호화되어 Crypto Footer에 저장
 - Crypto Footer: /data 파티션 끝 부분에 위치, 암호화 설정 정보 및 암호화된 DEK 저장 영역
- KEK 생성은 TEE 내부에서 수행되어 소프트웨어적으로 추출 불가
 - TEE (Trusted Execution Environment): OS와 독립된 하드웨어 영역으로, Trusty라는 전용 OS가 관리

■ 전체 암호화로 인한 부팅 시 앱 접근 제한 발생 → Android 10 이상 버전부터 FBE 사용

Android 정의

■ Android 보안 기능

■ FDE 암호화 동작 과정

- DEK 생성하여 AES-128-CBC 또는 AES-256-XTS 알고리즘으로 /data 파티션 암호화
- 사용자 인증수단 및 Salt 값 → Key 유도 함수 이용하여 IK (Intermediate Key) 생성
- TEE 내부에서 IK를 기기 고유 하드웨어 Key와 결합해 KEK 생성
- KEK를 통해 DEK 암호화 및 Crypto Footer에 저장

■ FDE 복호화 동작 과정

- 시스템 부팅과 함께 Crypto Footer 읽어 DEK 암호화 정보 확인
- 사용자 인증 수단 요구 → 입력 받은 정보 및 Salt 값 → Key 유도 함수 이용하여 IK 생성
- TEE 내부에서 KEK 생성 및 암호화된 DEK 복호화
- 복호화된 DEK를 통해 /data 파티션 복호화 → OS가 /data 파티션에 접근 가능

Android 정의

■ Android 보안 기능

■ FBE (File Based Encryption)

- Android 7.0 ~ 현재 버전에서 사용되는 /data 파티션 내 개별 파일 단위의 암호화 기능
 - Android 7.0: FBE 지원 시작
 - Android 10: FBE 사용 의무화
- 암호화 저장소 분리를 통해 파일별로 서로 다른 암호화 정책 적용 가능
 - CE (Credential Encrypted) 저장소: 사용자가 기기를 잠금 해제 한 후에만 접근 가능한 영역
 - DE (Device Encrypted) 저장소: 사용자의 기기 잠금 해제 여부와 관계 없이 접근 가능한 영역
- Direct Boot 지원
 - 시스템 부팅 시 DE 저장소가 복호화 → 잠금 해제 없이 알람, 전화, 긴급 문자 수신 가능
- Class Key (CE / DE Key), FEK (File Encryption Key)로 암호·복호화 수행

Android 정의

■ Android 보안 기능

■ FBE 암호화 동작 과정

- 파일 생성 시 해당 파일 전용 FEK 생성 및 파일 암호화
- 파일 저장 위치에 따라 CE 암호화 또는 DE 암호화 결정 → FEK를 CE Key 또는 DE Key로 암호화 (FEK)
- FEK를 파일 시스템의 i-node의 xattr 영역에 기록
 - xattr: 파일의 확장 속성으로 암호화 정책 및 Key 식별 정보 등 저장

■ FBE 복호화 동작 과정

- DE 저장소 복호화: 기기 부팅 → TEE 내부 하드웨어 키를 사용해 DE Key 복구 → DE Key를 통해 DE 영역의 암호화된 키 복호화하여 FEK 획득 → 획득한 FEK로 DE 영역 데이터 복호화
- CE 저장소 복호화: 사용자 인증수단 입력 요구 → 입력 받은 정보를 TEE로 전달하여 CE Key 복구 → CE 영역의 암호화된 키 복호화하여 FEK 획득 → 획득한 FEK로 CE 영역 데이터 복호화

Android 정의

■ Android 보안 기능

■ 보안부팅 (Secure Boot)

- 부팅 과정에서 실행되는 모든 소프트웨어가 신뢰할 수 있는 상태인지 단계별로 검증하는 부팅 방식
 - 모든 코드의 출처가 공격자 또는 손상된 코드가 아닌 신뢰할 수 있는 소스인지 확인
 - 변조된 OS, 악성 Bootloader, 취약점이 존재하는 Android 버전 실행 금지
- Chain of Trust 형성 → 부팅 과정의 각 단계에서 실행 처리 전 다음 단계의 무결성 검사
 - Hardware Root of Trust의 신뢰 요소를 통해 Chain of Trust가 가지고 있는 논리적 문제 해결
 - 논리적 문제: 'A → B → C → ... → 끝' 구조의 Chain에서 A를 누가 검증할 것인가에 대한 문제
- Rollback Protection을 통해 Android의 올바른 버전 확인 및 다운그레이드 방지

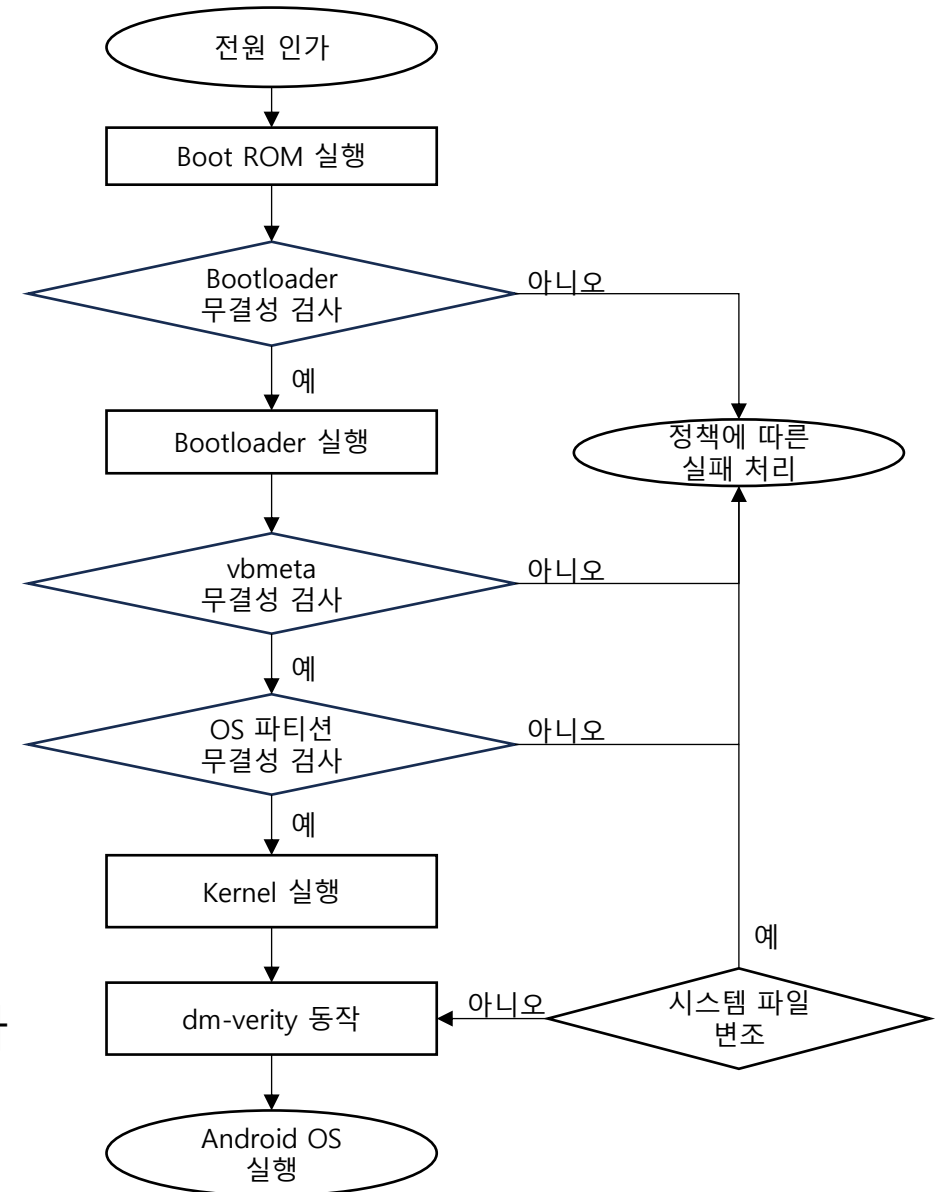
■ 암호학적 해시와 디지털 서명으로 무결성 검사 수행

Android 정의

■ Android 보안 기능

■ 보안부팅 (Secure Boot)의 동작 과정

- Boot ROM: Hardware Root of Trust 역할
 - Bootloader 위치 탐색, 이미지 로드, 무결성 검사
- Bootloader: Android OS 실행 전 작업 수행
 - 하드웨어 초기화, 커널 로드 전 무결성 검사 수행
- vbmeta: OS 파티션 정보에 대한 해시 값을 저장한 구조체
 - 해당 정보로 OS 이미지 변조되었는 지 확인
 - Rollback Index도 저장하여 Rollback Protection도 수행
- dm-verity: 부팅 후 시스템 파티션의 변조 여부 지속적 검사
 - Merkle hash tree 기반 해시 검증



Android 정의

■ Android 보안 기능

■ 권한

- 제한된 데이터 및 제한된 작업에 대한 접근을 보호해 사용자 데이터 보호 지원
- 기본적으로 SandBox 구조를 기반으로 모든 앱을 고립된 환경에서 실행
 - SandBox: 각 앱에 고유한 사용자 ID를 부여해 독립된 프로세스와 전용 디렉토리에 격리하여 앱 간 무단 데이터 접근과 시스템 자원 침해를 원천 차단하는 보안 모델
- 권한을 통해 앱 간 격리를 유지하며 시스템 자원 또는 사용자 데이터 접근 제어 가능
- Android 6.0 전 / 후 권한 변화: 설치 시 권한 일괄 승인 → 앱 실행 중 사용자 승인
- Android 10.0 전 / 후 권한 변화: 앱이 외부 저장소 전체 접근 가능 → 앱 전용 디렉토리 접근만 허용
- Android 11 ~ 14: 각 권한에 대해 세분화 강화

Android 정의

■ Android 보안 기능

■ 권한 유형

- 시스템에서 앱에 권한 부여 시 접근 할 수 있는 제한된 데이터 범위와 제한된 작업 범위 표시
- 권한의 보호 수준, 시스템 및 사용자 정보에 미치는 영향에 따라 5가지 유형으로 분류
 - 제조사 키로 서명된 시스템 앱은 Signature 권한과 Privileged 권한 동시 사용 가능
- 권한의 민감도에 따라 설치 시 자동 부여 되거나 실행 중 사용자 승인, 사용자 수동 설정을 통해 부여

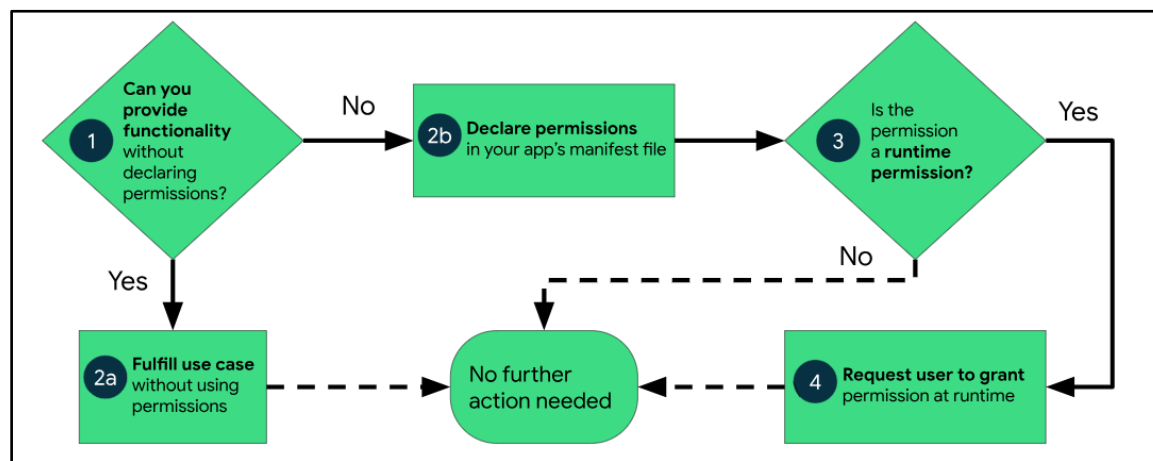
권한 유형	설명	특징
Normal 권한	사용자 정보나 시스템에 미치는 영향이 적은 권한	사용자가 별도 승인하지 않아도 앱 설치 시 자동 허용
Dangerous 권한	사용자 정보나 민감한 시스템 자원 접근 권한	Android 6.0 이후 실행 중 사용자 동의 필요
Signature 권한	동일한 인증서로 서명된 앱만 사용할 수 있는 권한	시스템 앱 또는 동일 제조사 앱 간 기능 공유에 사용
Privileged 권한	제조사 또는 시스템 앱에만 허용되는 권한	/system/priv-app만 요청 가능
Special 권한	시스템 설정 변경 등 매우 민감한 기능 접근 권한	일반 권한 요청 방식이 아닌 설정 메뉴에서 직접 허용

Android 정의

■ Android 보안 기능

■ 권한 동작 방식

- 권한 선언 없이 기능 제공 여부 확인 → 가능하면 권한 없이 작업 완료
- 불가능 시 AndroidManifest.xml에 권한 선언 및 해당 권한이 실행 중 사용자 승인 필요 여부 확인
- Signature, Privileged 권한은 시스템 인증을 통한 자동 부여
- Special 권한은 사용자 수동 설정으로 권한 부여



[Android 권한 동작 방식 워크플로우]

Android 정의

■ Android 보안 기능

■ 권한 유형별 대표 예시

- Android는 권한 유형 별로 시스템 기능 및 사용자 데이터 접근 권한을 구분하여 제공

권한 유형	대표적인 권한 예시	용도
Normal 권한	INTERNET, VIBRATE, SET_ALARM, ACCESS_WIFI_STATE	인터넷 접속, 진동 설정, 알람 설정, WIFI 상태 접근
Dangerous 권한	READ_CONTACTS, CAMERA, RECORD_AUDIO, READ_EXTERNAL_STORAGE, ACCESS_FINE_LOCATION	연락처 접근, 카메라 촬영, 녹음, 외부 저장소 접근, 위치 접근
Signature 권한	GOOGLE_AUTH, GOOGLE_APP_CERTIFICATE_ACCESS	Google 서명 앱 간 인증 토큰 공유, Google 앱 인증서 접근
Privileged 권한	BLUETOOTH_PRIVILEGED, INSTALL_PACKAGES	시스템 블루투스 제어, 앱 설치 관리
Special 권한	SYSTEM_ALERT_WINDOW, WRITE_SETTINGS, MANAGE_EXTERNAL_STORAGE	다른 앱 위에 그리기, 시스템 설정 변경, 외부 저장소 관리

데이터 수집 방법

MD-NEXT

■ 개요

- 모바일 기기에서 데이터 획득을 위한 포렌식 소프트웨어
- Android, iOS, Windows OS 등 다양한 운영체제 지원

■ 주요 기능

- 모바일 데이터 전체 / 선별 수집
- 다양한 해시 알고리즘 지원 → 증거 데이터 무결성 보장
- 암호화 파티션 자동 인식 및 복호화 지원
- PDF, XLS 등 보고서 생성



[MD-NEXT]

Rooting 기반 수집 방법론

■ Rooting이란?

- Android OS는 리눅스 기반 → 사용자 권한을 제한하는 방식으로 시스템 보안 유지
- Android에서 Root / Super User 권한을 획득하는 과정

■ 장점

- 전체 데이터 수집 가능: 접근 제한된 시스템 파일 등 수집 불가능했던 데이터 접근 가능
- 원활한 동적 분석: Root / Super User 권한을 기반으로 용이한 Android OS 제어

■ 단점

- 기기 안정성 하락: 잘못된 루팅 시 기기 사용 불가 상태로 전환
- 앱 실행 제한: 금융, 결제 앱 등 고신뢰성이 요구되는 앱 실행 차단

Rooting 기반 수집 방법론

■ Rooting 도구

이름	루팅 방식	설치 방식	지원 버전	특징
Magisk 	Systemless ¹⁾	APK 설치 → boot.img 패치	Android 6 ~	boot.img 패치 (Ramdisk ²⁾ 수정)
KernelSU 	Kernel - Based	커널 빌드 → anykernel3 ³⁾ 플래시	GKI 2.0 ⁴⁾ (Android 12 ~)	커널 소스 자체에 루팅 코드 삽입
Apatch 	Kernel - Based	APK 설치 → boot.img 패치	GKI 2.0 (Android 12 ~)	boot.img 패치 (커널 레벨에서 Kernel Patch Module 삽입)
KingRoot 	Exploit - Based	APK 설치	Android 2.3 ~ 7.0	시스템 취약점 이용 별도의 이미지 패치 X

1) Systemless: 시스템 파티션이 아닌 부팅 파티션을 수정해 루팅

2) Ramdisk: 부팅 시 커널이 가장 먼저 마운트하여 사용하는 임시 루트 파일 시스템

3) anykernel3: 커널 이미지를 기기에 안전하게 설치 (플래시) 할 수 있게 도와주는 스크립트 기반 툴킷

4) GKI 2.0: 구글이 도입한 표준화된 커널 이미지 규격 → 공통 커널을 통해 유지 보수 비용 절감 (Android 12 이후 도입)

Rooting 기반 수집 방법론

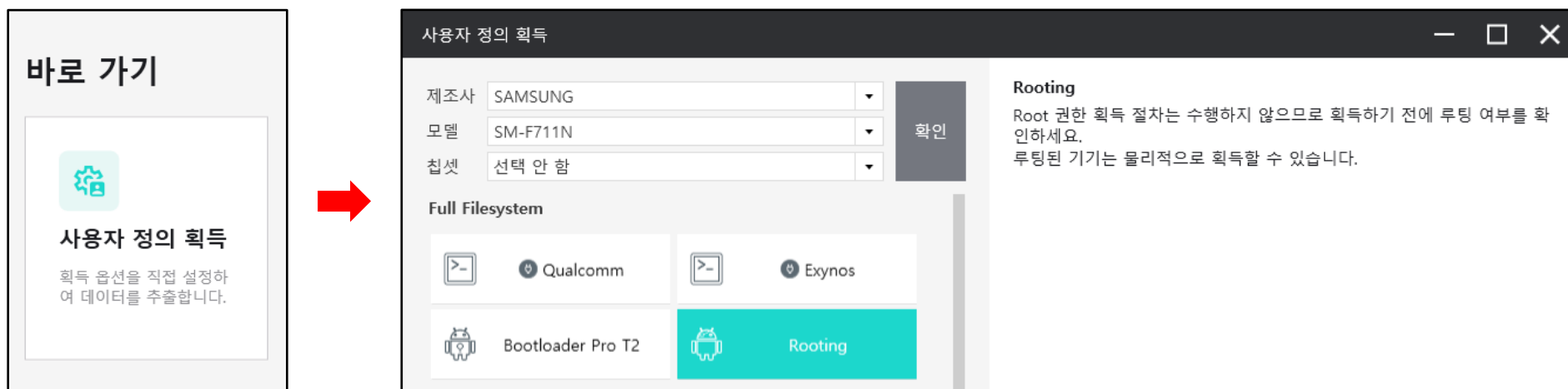
■ MD-NEXT에서의 Rooting 활용

■ 사용자 정의 획득 기능 이용

- 획득 옵션을 직접 설정하여 데이터를 추출하는 방식 → 사전에 루팅된 기기 필요

■ Rooting 기기의 경우 Full Filesystem / Physical 방식 추출 가능

- Full Filesystem: 비할당 영역을 제외한 모든 파일 시스템 영역 추출 방식
- Physical: 비할당 영역을 포함한 추출 방식

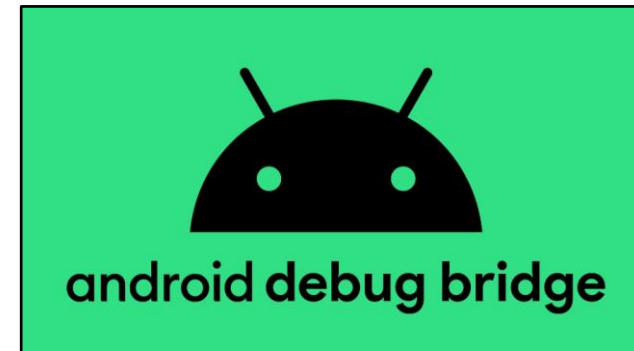


[사용자 정의 획득 방식 - Rooting 옵션 지정]

Backup 기반 수집 방법론

■ ADB (Android Debug Bridge)

- Android 기기와 컴퓨터 간 연결 및 디버깅 작업 등을 위한 다목적 명령줄 (CLI) 도구



[Android Debug Bridge 로고]

■ ADB 구성요소

- 서버 (Server): PC에서 백그라운드로 실행되며 클라이언트와 ADB 데몬 사이 통신 관리
- 클라이언트 (Client): 명령을 보내는 주체 (ex. PC, 터미널 등)
- ADB 데몬 (adbd): 각 기기 내부에서 백그라운드로 실행되며 명령을 실제 수행하는 주체

■ ADB 특징

- 비루팅 모바일 데이터 수집 가능 → /system 등 일부 파티션은 루팅이 필수, 일부 접근 제한
- 실시간 앱 로그 수집 가능

Backup 기반 수집 방법론

■ ADB Backup

- Android 기기 데이터를 PC로 백업하는 기능
 - APK, /data/data/<package>, 공유 스토리지, 시스템 설정 등 백업
 - 출력 파일은 .ab 파일 → tar 변환 후 압축 해제하여 backup 데이터 획득
 - ABE (Android Backup Extractor) 도구로 tar 변환 수행
- AndroidManifest.xml에서 android:allowBackup이 true여야 사용 가능
- 동작 과정
 - adb backup 요청을 adbd로 전달
 - adbd은 요청 수신 후 backup 서비스 핸들러 전달
 - BackupManagerService 호출 및 backup 세션 초기화
 - backup 대상 앱 확인 후 backup 프로세스 동작

옵션	설명
-apk	APK 파일 포함 추출
-shared	/sdcard 포함 추출
-all	앱 데이터 추출
-system	시스템 앱 포함
-f	출력 파일 지정

Backup 기반 수집 방법론

■ MD-NEXT에서의 ADB Backup 활용

- 최근 Android APK는 android:allowBackup을 false로 설정하는 추세
- MD-NEXT는 APK와 앱 데이터가 서로 다른 영역에 저장한다는 점을 이용
 - APK: /data/app에 저장, 앱 데이터: /data/data/<package>에 저장
- MD-NEXT는 android:allowBackup을 true로 설정한 구버전 APK로 다운그레이드
 - 사용자가 옵션 설정한 경우에 다운그레이드 동작, 미설정 시 기존 APK 대상으로 Backup 수행
 - 동작 과정
 - adb install -r -d 명령어로 다운그레이드 및 재설치
 - 다운그레이드 후 adb backup 수행
 - backup 완료 후 기존 버전 앱으로 복구

Backup 기반 수집 방법론

■ Samsung Smart Switch

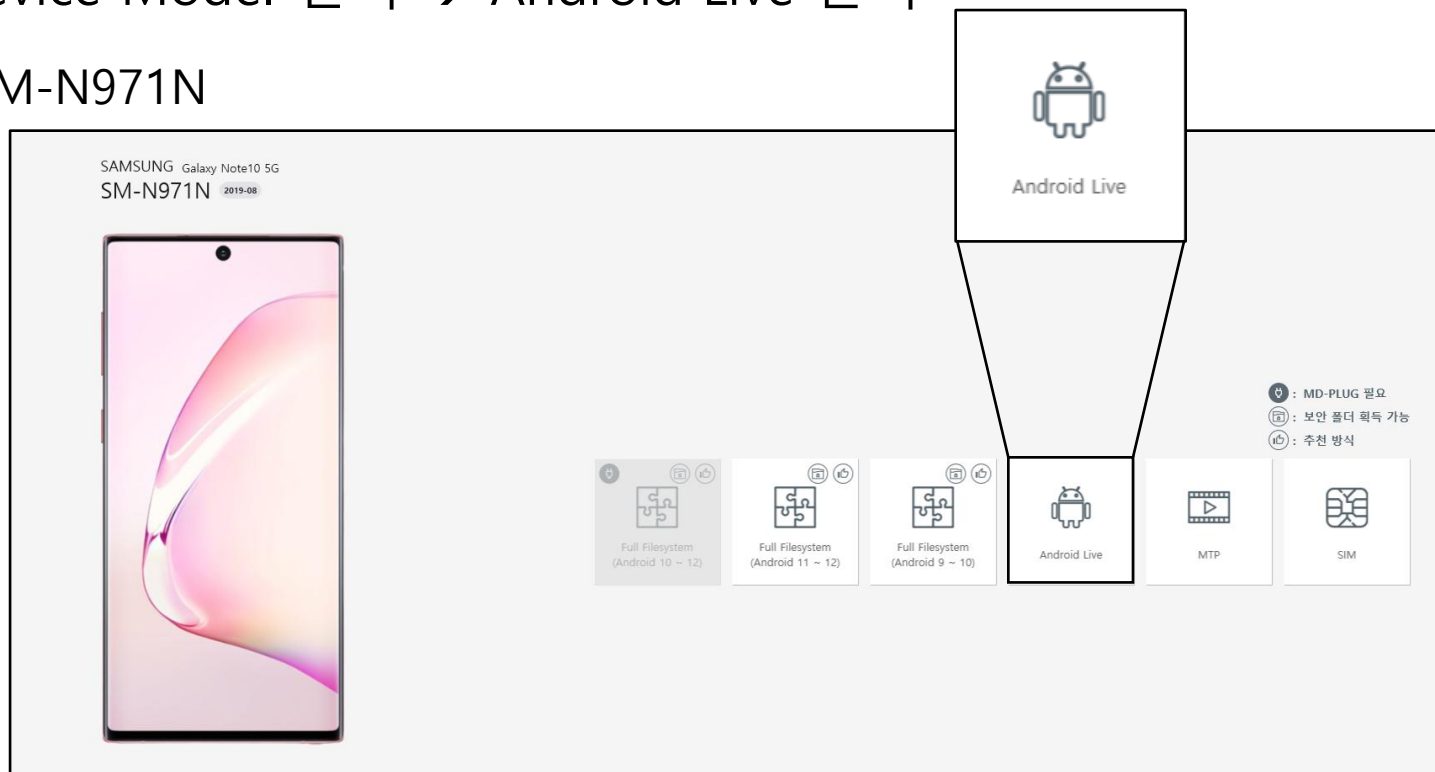
- 기존 Android, iOS에 저장된 데이터를 Samsung 기기로 백업 및 전송하는 솔루션
- 기기 데이터 백업 → 컴퓨터 / 클라우드 등 저장 → 필요시 복원
- 유선 및 무선 연결 지원
- 지원 기기
 - 스마트폰 / 태블릿
 - Android 4.3 이상 / iOS 5 이상 / BlackBerry OS 7 or 10 / Windows Mobile 8.1 or 10
 - PC (백업 및 복원)
 - Windows 10 / macOS X 10.9 이상



Backup 기반 수집 방법론

■ Android Live

- MD-NEXT의 Live 기반 모바일 데이터 수집 기능
- 수집하려는 Device Model 선택 → Android Live 클릭
 - 분석 모델: SM-N971N



[MD-NEXT / Android Live 선택 화면]

Backup 기반 수집 방법론

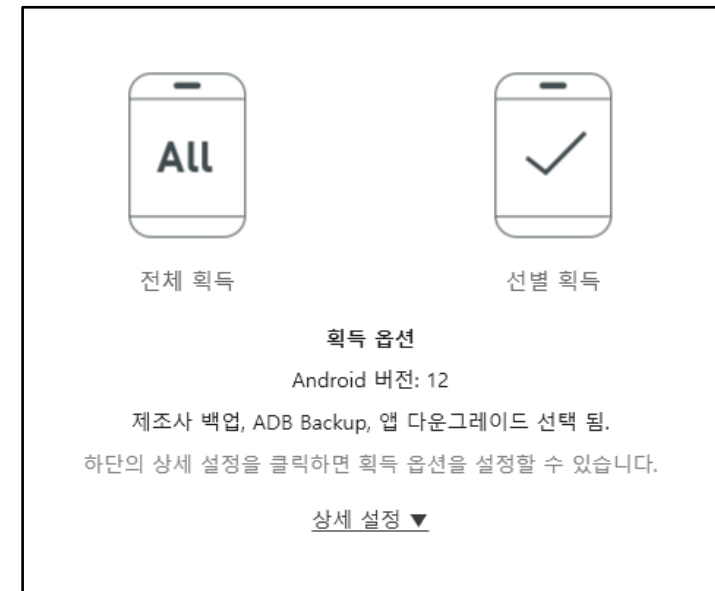
■ Android Live

■ 획득 옵션

- 전체 획득: 접근 가능한 모든 파일을 수집
- 선별 획득: 필요한 파일만 선택 수집

■ 상세 설정

- 제조사 백업: 제조사에서 제공한 백업 기능 사용
 - 해당 실습에서는 Samsung Smart Switch 이용
- 획득 앱 설치: 기기에 앱을 설치하여 획득
 - 제조사 백업 기능이 없는 경우 MD-NEXT 자체 앱 설치 후 수집하는 기능
 - Samsung Smart Switch 지원 가능하므로 옵션 설정 X
- ADB 백업: Android에서 제공하는 기능 사용



[MD-NEXT / 전체 획득, 선별 획득 선택 화면]

Backup 기반 수집 방법론

■ Android Live

■ 데이터 추출

- 완료 후 .mdf 파일과 함께 SHA256 해시 값 제공 → 무결성 검증 가능

획득 중

획득 중

경과 시간 : 00:00:02

업 (0/217) 파일 (0/1)

SYSTEM_BACKUP

/system_backup/shell/dumpsys/package/com.google.android.gms.out

획득 대상 (0/217)

획득 제외 (0)

네트워크 로그

기타

연락처

메모

알림

메시지

미디어 로그

브라우저

이메일

시스템 정보

Android 시스템

android

ADW/NEAVIS ICT

획득 대상 (0/1)

획득 제외 (0)

media

/media

획득 검증

획득 파일 폴더 : E:\WeeeeWme

획득 목록

me.mdf

완료

파일시스템

LOGICAL (폴더 : 645, 파일 : 1,109)

해시

SHA256 D5155CA4E7DE125B17A2BC9BA2D24E8431F904BCDCFF4B75E7AED14766A303C5

파일시스템

LOGICAL (폴더 : 645, 파일 : 1,109)

해시

SHA256 D5155CA4E7DE125B17A2BC9BA2D24E8431F904BCDCFF4B75E7AED14766A303C5

검증 완료

1 / 1 (성공 : 1, 실패 : 0, 취소 : 0)

취소

닫기

[MD-NEXT / 추출 완료 화면]

데이터 분석 방법

Android 데이터 종류 및 형태

■ Android 데이터 분류

데이터 분류	항목	예시
시스템 정보	기기 모델 및 OS 버전 정보	기기 모델명 (SM-S928N), Android 버전 (14) 등
	네트워크 연결 정보	연결된 Wi-Fi SSID*, Bluetooth 페어링 이력 등
	다운로드 및 설치된 앱 목록	설치된 패키지 명, 앱 설치 및 업데이트 시각 등
	디바이스 설정	회전 방향, 화면 밝기, 비행기 모드 설정 등
사용자 정보	연락처 및 통신 정보	수신 / 발신 / 부재중 기록, 전화번호 등
	문자 메시지 정보	SMS, MMS 등
	사용자 계정 정보	Google 계정, 삼성 계정 등
	멀티미디어 파일	사진, 동영상, 오디오 파일 등
	사용자 설정 정보	배경화면 설정, 테마 설정 정보 등
어플리케이션 사용 정보	앱 버전 및 설정 정보	설치된 앱 버전, 앱 알림 설정 여부 등
	앱 별 권한	위치, 카메라, 연락처 접근 권한 허용 여부 등
	앱 데이터	메신저 대화 내역, Samsung Note 데이터 등

* Wi-Fi SSID: Service Set Identifier의 약어, Wi-Fi 무선 네트워크 이름을 지시 (ex. KT_GiGA_WiFi)

Android 데이터 종류 및 형태

■ Android 데이터 형태

- 정형 데이터: 고정된 스키마가 있는 테이블 구조 데이터
- 반정형 데이터: Key - Value 형태, 태그 기반 등 스키마가 존재하지만 테이블 구조는 아닌 데이터
- 비정형 데이터: 구조가 없는 형식의 데이터
- 바이너리 / 미디어 데이터: 사진, 동영상 등 바이너리의 형태로 저장된 데이터

구분	정형 데이터	반정형 데이터	비정형 데이터	바이너리 / 미디어 데이터
저장 형태	테이블 구조	Key - Value, 태그 등	텍스트 등	바이너리 형태
파일 예시	SQLite DB 등	XML, JSON 파일 등	txt 형식의 로그 파일 등	JPG, MP4, MP3 등
Android 사용 예시	/data/data/[패키지명]/databases 등 앱 별 데이터베이스 저장	/data/system/packages.xml 등 설치된 앱 목록 확인	/cache/recovery/last_log 등 recovery mode 최근 로그 기록	/sdcard/DCIM/Camera/ 등 사진 데이터 저장

스마트폰 기본 정보 분석

■ 기기 정보 / 시스템 로그 분석

- 기기 정보 및 시스템 로그 데이터 분석 시 얻을 수 있는 정보
 - 사용자 계정 정보
 - 기기 사용 시간
 - GPS 등 위치 정보
 - 설치 / 삭제된 앱
 - Wi-Fi, Bluetooth 등 통신 연결 정보
 - 공장 초기화 여부
- 제조사, 안드로이드 버전 등 경로에 차이가 존재할 수 있음
- 기기 정보 및 시스템 로그는 사용자 접근이 어려운 영역에 존재 → 포렌식적으로 유의미한 데이터 획득 가능

스마트폰 기본 정보 분석

■ 기기 정보 / 시스템 로그 데이터 경로

- Samsung Galaxy Z Flip 3 5G (SM-F117N, Android 15)

분류	저장 정보	파일 경로
커널	마지막 부팅에서 발생한 커널 로그	/proc/last_kmsg
	현재 발생 중인 커널 로그	/proc/kmsg
	커널 빌드 정보	/proc/version
	커널 부팅 시 전달되는 파라미터	/proc/cmdline
전원	전원 종료 로그	/data/log/shutdown_profile.[%number%].txt
	재시작 로그	/data/log/power_off_reset_reason.txt
	최초 부팅 로그	/data/log/Firstbootlog.txt
에러	앱 사용 중 발생한 에러 로그	/data/system/dropbox/system_server_wtf@[%Timestamp%].txt

스마트폰 기본 정보 분석

■ 기기 정보 / 시스템 로그 데이터 경로

■ /proc/version: 커널 버전 정보

- 사용 중인 커널 버전: Linux version 5.4.274
- 기기 빌드 식별자: abF711NKSSAKYF1 (F711N은 Samsung Galaxy Z Flip 3 모델 코드)
- 사용된 컴파일러: clang version 11.0.2
- 사용된 링커: LLD 11.0.2

```
Linux version 5.4.274-qgki-30959342-abF711NKSSAKYF1 (dpi@21DJ6908) (Android (6877366 based on r383902b1) clang version 11.0.2 (https://android.googlesource.com/toolchain/llvm-project b397f81060ce6d701042b782172ed13bee898b79), LLD 11.0.2 (https://android.googlesource.com/toolchain/llvm-project b397f81060ce6d701042b782172ed13bee898b79)) #1 SMP PREEMPT Wed Jun 4 15:12:13 KST 2025
```

[/proc/version]

47

스마트폰 기본 정보 분석

■ 기기 정보 / 시스템 로그 데이터 경로

■ /data/log/shutdown_profile.[%number%].txt: 전원 종료 로그

- Start shutdown radios: 무선 통신 종료 시작
- wait for animation end: 전원 종료 영상 대기
- reboot requested: 재부팅 요청 발생
- Rebooting: 재시작

◦ 해당 전원 종료 로그는 다시 시작 절차임을 확인 가능

```
06-04 14:26:31.320 ShutdownThread: !@Start shutdown radios
06-04 14:26:31.330 ShutdownThread: Waiting for Radio...
06-04 14:26:31.331 ShutdownThread: !@before sleep
06-04 14:26:31.431 ShutdownThread: !@after sleep
06-04 14:26:31.431 ShutdownThread: !@[Phone off retry:181534] : 193422 radio=false
06-04 14:26:31.432 ShutdownThread: !@Radio turned off.
06-04 14:26:31.432 ShutdownThread: !@waitForAnimEnd
06-04 14:26:31.433 ShutdownThread: !@wait for animation end
06-04 14:26:31.642 ShutdownThread: !@run, reboot requested reason= (uid: 0 pid: 17619 processName: app_process)
06-04 14:26:31.642 ShutdownThread: !@Rebooting, reason: (uid: 0 pid: 17619 processName: app_process)
```

[shutdown_profile.1.txt 일부]

스마트폰 기본 정보 분석

■ 기기 정보 / 시스템 로그 데이터 경로

- Samsung Galaxy Z Flip 3 5G (SM-F117N, Android 15)

분류	저장 정보	파일 경로
네트워크	데이터 사용량 정보 (네트워크 인터페이스 별)	/data/system/netstats/dev.[%Start Timestamp%]-([%End Timestamp%])
	데이터 사용량 정보 (애플리케이션 별)	/data/system/netstats/uid.[%Start Timestamp%]-([%End Timestamp%])
	데이터 사용량 정보 (시스템 전체)	/data/system/netstats/xt.[%Start Timestamp%]-([%End Timestamp%])
	연결한 Wi-Fi 정보 (SSID, 암호화 방식)	/data/system/WifiConfigStore.db
		/data/system/WifiHistory.db
	Bluetooth 설정 정보	/data/misc/bluedroid/bt_config.conf
		/data/misc/bluedroid/bt_config.bak

스마트폰 기본 정보 분석

■ 기기 정보 / 시스템 로그 데이터 경로

■ 네트워크 (Wi-Fi)

- /data/system/WifiConfigStore.db: 현재 저장된 Wi-Fi 설정 정보
 - "DFRC"WPA_PSK: "DFRC"에 WPA_PSK 방식으로 연결
 - WPA_PSK: 접속 시 공유된 비밀번호 (PSK)를 이용해 인증하는 방식 / WPA2
 - "DFRC"SAE: "DFRC"에 SAE 방식으로 연결
 - SAE: Password - Authenticated Key Exchange를 이용해 인증하는 방식 / WPA3
 - 같은 SSID ("DFRC")에 두 개 인증 방식 → 공유기 WPA2 / WPA3 혼합 모드

	_ID	CONFIG_KEY	CREATION_TIME	PERSONALIZED_CONN_OPTION
	...	필터	필터	필터
1	1	"DFRC"WPA_PSK	1749016703502	0
2	2	"DFRC"SAE	1749016712733	0

[WifiConfigStore.db]

스마트폰 기본 정보 분석

■ 기기 정보 / 시스템 로그 데이터 경로

■ 네트워크 (Bluetooth)

- /data/misc/bluedroid/bt_config.conf: Bluetooth 설정 정보 저장

- LmpVer = 11 (Bluetooth 5.2) / Address (Bluetooth MAC Address) / BLE (Bluetooth Low Energy) 보안 키
- 기기 모델명 / ScanMode = 1 (검색 불가능, 페어링 가능) / DiscoveryTimeout (탐색 시간, 120초)

```
[Adapter]
Manufacturer = 29
LmpVer = 11
LmpSubVer = 31390
Address = f0:f5:64:d2:31:2b
LE_LOCAL_KEY_IRK = 4bebbc457170b1eeda9a4fdcb99db12f99599a41a3c4b8aff9286f938a9554b3
LE_LOCAL_KEY_IR = 70de8b93ffcd15a2d59445621dd522de5f8facc6319bbd30eabbdece2725bdd3
LE_LOCAL_KEY_DHK = bfc9ebbc9865661e8702bbe5b81c616db256d6d0edcb9af807dbddfd88468fe5
LE_LOCAL_KEY_ER = 03244889d76dee0a3dcc691bf1d1d09d793b1cd9880d0b34d58495af92046e34
Name = Galaxy Z Flip3 5G
ScanMode = 1
DiscoveryTimeout = 120
```

[bt_config.conf 일부]

스마트폰 기본 정보 분석

■ 기기 정보 / 시스템 로그 데이터 경로

- Samsung Galaxy Z Flip 3 5G (SM-F117N, Android 15)

분류	저장 정보	파일 경로
애플리케이션 관련	설치된 애플리케이션 기본 정보 (패키지 명, 실행 경로, UID 등)	/data/system/packages.list
	설치된 애플리케이션 상세 정보 (APK 경로, 설치 시간, 권한, 서명 정보 등)	/data/system/packages.xml
	Google 관련 애플리케이션 데이터 및 보안 검사 데이터베이스	/data/data/com.google.android.gms/database/gass.db
	자동 계정 인증이 등록된 애플리케이션 목록	/data/system/users/0/registered_services/android.accounts.AccountAuthenticator.xml
	자동 동기화 애플리케이션 목록	/data/system/users/0/registered_services/android.content.SyncAdapter.xml
	자동 동기화 애플리케이션 및 계정 기록	/data/system_de/0/accounts_de.db
		/data/system_ce/0/accounts_ce.db
	애플리케이션 별 실행 횟수 (실행 / 중지, Timestamp)	/data/system/dmappmgr.db

스마트폰 기본 정보 분석

■ 기기 정보 / 시스템 로그 데이터 경로

- /data/system/packages.list: 설치된 어플리케이션 기본 정보
 - 패키지명: com.samsung.android.themestore (삼성 테마 스토어 앱)
 - UID: 10145
 - UID는 시스템 내 앱의 고유 번호 의미
 - 설치 경로: /data/user/0/com.samsung.android.themestore
 - 속성: samsung:privapp (삼성 전용 앱) / targetSdkVersion=35 (타겟 Android 버전) / partition=system (시스템 파티션에 포함)
 - 앱 소유자: @system (루트 권한으로 관리하는 앱 의미)

```
com.samsung.android.themestore 10145 0 /data/user/0/com.samsung.android.themestore samsung:privapp:targetSdkVersion=35:partition=system 3003 0 600310103 1 @system
```

[packages.list 일부]

스마트폰 기본 정보 분석

■ 기기 정보 / 시스템 로그 데이터 경로

■ /data/data/com.google.android.gms/databases/gass.db

- Google 관련 애플리케이션 데이터 및 보안 검사 데이터베이스
- 패키지명 / 버전 코드 / SHA256 해시 값 → 애플리케이션 무결성 확인용

	_id	pb	package_name	version_code	digest_sha256
...	필터	필터	필터	필터	필터
1	1	BLOB	com.topjohnwu.magisk	1	5a6fc51f1f74647e0b31b50a7e8694f2c73...
2	2	BLOB	com.topjohnwu.magisk	30600	flffc3c9a5614c251ba6bada308163acc3c...
3	3	BLOB	com.skt.nugu.apollo	4030127	e538109b0db15f131a023a651747c1dfb7b...
4	4	BLOB	kr.co.captv.pooqV2	2026022505	921f8e551ca1125da93a5faa2940776b809...
5	5	BLOB	skplanet.musicmate	205407	615120aaee0a9d62798463fd1603fa7c279...
6	6	BLOB	com.skt.tmap.ku	3101	c2256f5513f3949b8ed3746bb0c620dc530...
7	7	BLOB	com.android.vending	85032730	9fb3f3b1b3be84c935c212581940d16e3c9...
8	8	BLOB	com.elevenst	103301	0d1c89ca20db8e88584126638936abdd9c4...
9	9	BLOB	com.sktelecom.vcoloring	2020101	f0ebe24d781283fb78837d128f6e665da09...
10	10	BLOB	com.sktelecom.tauth	136	54f3e1a3450d2683bd5273ae473a6c61043...
11	11	BLOB	android.autoinstalls.config.samsung	1001	7bb39962a89243d793f07d327dd72e3d5cb...
12	12	BLOB	com.nhn.android.search	12192504	811083e6fd46f83395b839f1635e47738ff...
13	13	BLOB	com.google.android.contactkeys	14255	6c0c636520587745297afa7d9fba320f6ec...
14	14	BLOB	com.google.android.youtube	1552287168	aaec5090f148a5c7a33684779dd29e561a3...
15	15	BLOB	com.google.android.safetycore	14350	2e2a5b6a630f0ed6252ba78c5d3fedba218...

[gass.db]

스마트폰 기본 정보 분석

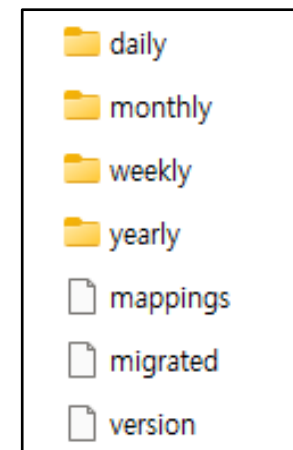
■ 기기 정보 / 시스템 로그 데이터 경로

■ Usagestats

- Android에서 연, 월, 주, 일 별 애플리케이션 사용 통계 기록
- 경로: /data/system_ce/0/usagestats (Samsung Galaxy Z Flip 3, Android 15 버전 기준)

■ Android 버전별 Usagestats 정리

구분	Android 5 ~ 9	Android 10	Android 11 ~
형식	XML	Binary	Binary
경로	/data/system/0/usagestats	/data/system_ce/0/usagestats	/data/system_ce/0/usagestats
디렉터리 이름	daily, weekly, monthly, yearly	daily, weekly, monthly, yearly	daily, weekly, monthly, yearly
파일 이름 형식	Timestamp (Unix Milliseconds)	Timestamp (Unix Milliseconds)	Timestamp (Unix Milliseconds)
메타데이터	version	version	version, migrated, mappings
version	3	4	5



[/data/system_ce/0/usagestats]

사용자 애플리케이션 데이터

■ 애플리케이션 데이터 분석 필요성

- 사용자의 애플리케이션 사용 기록 확보 → 포렌식 분석을 통해 법적 증거 형성 가능

■ 애플리케이션 종류

- 사용자 애플리케이션: 사용자가 별도로 설치한 애플리케이션
 - 설치 경로: /data/app/[인코딩 된 문자열]/*.apk
 - 예시: 카카오톡, 인스타그램 등
- 기본 애플리케이션: 기기 출고 시, 기본으로 제공되는 애플리케이션
 - 설치 경로: /system/app/*.apk, /system/priv-app/*.apk
 - 예시: 계산기, 전화 앱 등

사용자 애플리케이션 데이터

■ 애플리케이션 데이터 파일 추출

- *.db, *.xml 등 다양한 형식의 파일 존재
- 사용자 애플리케이션 별 디렉터리 구조와 데이터 저장 방식 상이

■ 기본 설치 앱 및 대표 애플리케이션 경로 예시

- /data/data/[패키지 명] 혹은 /data/user_de/0/[패키지 명] 하위 경로에 존재

이름	파일 경로
메시지	/data/data/com.android.providers.telephony/databases/mmssms.db
연락처	/data/data/com.samsung.android.providers.contacts/databases/contacts2.db
통화 내역	/data/data/com.samsung.android.providers.contacts/databases/calllog.db
캘린더	/data/data/com.android.providers.calendar/databases/calendar.db
이메일 (G-mail)	/data/data/com.google.android.gm/databases/EmailProvider.db

사용자 애플리케이션 데이터

■ 메시지

- 경로: /data/data/com.android.providers.telephony/databases/mmssms.db
- 확인 가능 정보
 - 수신자 전화번호
 - 전송 날짜
 - SMS 내용
 - 사용 메시지 앱

_id	address	date	date_sent	read	status	type	body	error_code	sub_id	creator
...	필터	필터	필터	필터	필터	필터	필터	필터	필터	필터
1	01087730416	1749016680000	0	1	-1	5	안녕하세요	0	-1	com.samsung.android.messaging
2	01087730416	1749016684000	0	1	-1	5	감사해요	0	-1	com.samsung.android.messaging

[mmssms.db]

사용자 애플리케이션 데이터

■ 캘린더

- 경로: /data/data/com.android.providers.calendar/databases/calendar.db

- 확인 가능 정보

- 사용자 설정 일정 → My Calendar[타임스탬프] 형식의 _sync_id 생성 (로컬 DB와 서버를 동기화하기 위한 ID)

- 일정 내용

- 일정 시작일 → Unix Timestamp (Milliseconds)

- 일정 날짜

_id	_sync_id	dirty	mutators	title	dtstart	dtend
필터	필터	필터	필터	필터	필터	필터
728	My calendar1773038590927	1	com.samsung.android.calendar	초안제출	1773100800000	1773187200000
727	My calendar1773038583086	1	com.samsung.android.calendar	3주차 안드로이드 발표	1773360000000	1773446400000
12	NULL	NULL	NULL	19대 대통령선거	1494288000000	1494374400000
13	NULL	NULL	NULL	20대 대통령선거	1646784000000	1646870400000

[calendar.db 일부]

사용자 애플리케이션 파일 (APK) 분석

■ APK 분석 필요성

- 악성 앱 분석: 악성 앱의 동작 과정 및 정보 탈취, 원격 제어 등 악성 행위 파악
 - 스미싱 앱 분석을 통한 개인정보 유출 매커니즘 파악
- 취약점 진단: 개발 과정에서 파악하지 못한 취약점 진단 및 개선 방안 도출
 - 하드 코딩된 API 키, 인증 정보 노출, 불필요한 권한 요청 등
- 앱 동작 및 구조 이해: APK 내부 구성 요소 및 실행 코드 분석을 통한 앱 기능 파악
 - 메신저 앱의 암호화 알고리즘 / 암호키 파악 → 법적 증거로 이용 가능한 메신저 대화 내용 확보

■ 기본 애플리케이션은 삭제 불가

- /system/app/ 경로 설치 앱: 부팅 시 Read-Only로 마운트 / Privileged Permission* X
- /system/priv-app/ 경로 설치 앱: 부팅 시 Read-Only로 마운트 / Privileged Permission O

* Privileged Permission: 일반 앱은 사용할 수 없는 시스템 타임 설정, 통신 제어, 저장소 접근 관리 등 OS 수준의 작업을 수행할 수 있도록 하는 특수 권한

사용자 애플리케이션 파일 (APK) 분석

■ APK 구조

■ AndroidManifest.xml

- 앱의 패키지명, 버전, 컴포넌트, 권한 등의 정보 포함
- 부팅 시 Android 시스템이 가장 먼저 읽는 파일
- Binary XML로 변환 → 데이터 확인을 위해 AXMLPrinter2 등 도구

■ classes.dex

- 안드로이드 가상 머신 Dalvik / ART가 인식할 수 있도록 생성된 파일
- Java, Kotlin 등으로 작성된 소스 코드 → dex 포맷으로 컴파일

■ META-INF

- 서명 및 무결성 검증 정보 저장 디렉토리

AndroidManifest.xml	
classes.dex	assets/
META-INF	lib/
resources.arsc	res/

사용자 애플리케이션 파일 (APK) 분석

■ APK 구조

■ resources.arsc

- 빌드 과정에서 생성된 리소스 ID의 매핑 테이블이 저장된 파일
 - 리소스: 문자열, 색상, 스타일, 레이아웃 등 앱에서 사용하는 데이터

■ assets/

- HTML, JSON 등 앱에서 사용하는 추가 파일 저장 디렉토리
- 컴파일 되지 않은 원본 형태로 저장

■ lib/: C / C++ 등으로 작성된 네이티브 라이브러리 (.so) 저장 디렉토리

■ res/

- 앱 UI 및 리소스 파일이 저장되는 디렉터리
- 빌드 과정에서 각 리소스에 ID 부여 → 해당 ID는 resources.arsc에 저장

AndroidManifest.xml	
classes.dex	assets/
META-INF	lib/
resources.arsc	res/

사용자 애플리케이션 파일 (APK) 분석

■ APK 분석 실습

■ 실습 환경

- jadx-gui-1.5.5

■ 실습 목적

- APK 구조 관찰 및 파일 내부 정보 확인

■ 분석 대상

- Coloring for Kids (org.androidsoft.coloring_2.apk)
 - 어린이용 색칠 놀이 앱



[jadx-gui 로고]



[jadx-gui에서 확인한 앱 리소스 목록]

사용자 애플리케이션 파일 (APK) 분석

■ APK 분석 실습

■ AndroidManifest.xml

- manifest 태그
 - versionCode: 2 / versionName: 1.0.1
 - package: org.android.coloring
- uses-sdk 태그
 - minSdkVersion: 11 / targetSdkVersion: 19
- uses-permission 태그
 - android.permission.WRITE_EXTERNAL_STORAGE
 - 외부 저장소에 파일을 쓰거나 수정할 수 있도록 허용하는 권한

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    android:versionCode="2"
    android:versionName="1.0.1"
    package="org.androidsoft.coloring">
    <uses-sdk
        android:minSdkVersion="11"
        android:targetSdkVersion="19"/>
    <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
</manifest>
```

[AndroidManifest.xml 일부]

사용자 애플리케이션 파일 (APK) 분석

■ APK 분석 실습

■ classes.dex

- dex 파일은 사람이 읽기 어려운 바이너리 형태 → jadx-gui에서 dex 파일을 자동으로 java 코드 변환 후 제공
- 앱의 실제 동작 로직 파악 가능
 - coloring: 색칠 앱의 기능 관련 패키지 (색 선택, 메인 색칠 등)
 - utils: 색칠 앱 화면에서 이동으로 피오르 하는 리스스 / 이미지 로드 기능 관련 패키지

	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
0000	64	65	78	0A	30	33	35	00	8B	4D	69	C4	DE	F1	C0	1F
0010	C4	99	8D	FB	97	D1	33	6E	27	F2	D2	8E	DB	55	BF	B3
0020	3C	E8	00	00	70	00	00	00	78	56	34	12	00	00	00	00
0030	00	00	00	00	6C	E7	00	00	08	04	00	00	70	00	00	00
0040	C5	00	00	00	90	10	00	00	88	00	00	00	A4	13	00	00
0050	09	01	00	00	44	1C	00	00	10	02	00	00	8C	24	00	00
0060	3D	00	00	00	0C	35	00	00	90	AB	00	00	AC	3C	00	00
0070	24	83	00	00	26	83	00	00	29	83	00	00	2C	83	00	00
0080	32	83	00	00	35	83	00	00	3F	83	00	00	47	83	00	00

[classes.dex 내용]



소스코드
org.androidsoft
coloring
ui
util
BuildConfig
R
utils
credits
res
ui
CommonConstants

[classes.dex 디컴파일 결과]

사용자 애플리케이션 파일 (APK) 분석

■ APK 분석 실습

■ resources.arsc

- 빌드 과정에서 생성된 리소스 ID의 매핑 테이블이 저장된 파일 → '@type/name' 형식으로 빠르게 접근 가능

- type: 리소스 종류

- name: 리소스 이름

- id: 리소스 고유 식별자

```
<public type="drawable" name="background" id="0x7f020000" />
<public type="drawable" name="background_land" id="0x7f020001" />
<public type="drawable" name="icon" id="0x7f020002" />
<public type="drawable" name="outline001_balloons" id="0x7f020003" />
```

[resources.arsc 내 ID 테이블]

```
<application
    android:label="@string/app_name"
    android:icon="@drawable/icon"
```

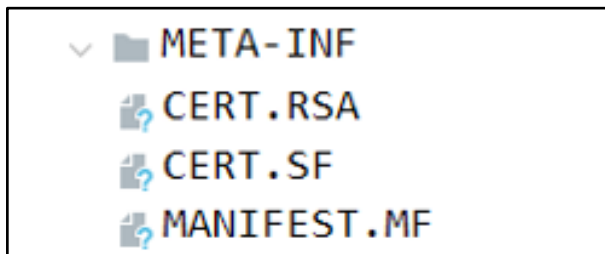
[AndroidManifest.xml – resources.arsc 사용 예제]

사용자 애플리케이션 파일 (APK) 분석

■ APK 분석 실습

■ META-INF

- 서명 및 무결성 검증 관련 정보 및 파일을 저장하고 있는 디렉토리
 - MF: 앱 내 모든 파일 (png, xml 등) SHA1 해시 값 목록 저장
 - SF: MF 파일 각 항목을 해시 값을 다시 해시한 값 저장 → MF 파일 자체의 변조 여부 확인
 - RSA: 개발자 인증서 및 서명 정보 저장



[META-INF 폴더]

```
Manifest-Version: 1.0
Created-By: 1.8.0_232 (Private Build)

Name: res/drawable-mdpi-v4/outline007_flowers.png
SHA1-Digest: dG/i2yBt64Hd3q/f00diaz+ouYc=
```

[MANIFEST.MF 파일 내용]

사용자 애플리케이션 파일 (APK) 분석

■ APK 분석 실습

■ res

- 앱에서 사용되는 UI 설정 및 리소스들이 저장된 폴더
 - drawable - hdpi / ldpi / mdpi - v4: 앱에서 사용하는 아이콘, 배경라운드 이미지 등을 해상도 별로 나눠 저장
 - layout: 화면 UI 및 레이아웃 관련 xml 파일 저장 (ex. 색칠 팔레트 넓이 / 높이 등)

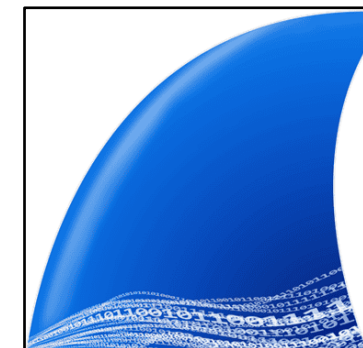


그 외 분석 방법론

■ 동적 분석

■ 네트워크 분석

- 서버 기반 모바일 앱 분석 시 필요
- 서버와 통신하는 API 요청 및 HTTP / HTTPS 통신 분석
 - 로그인 요청, 업로드 파일, 암호화되지 않은 데이터 등 확인 가능
- 대표 도구: Burp Suite, Wireshark



[Burp Suite 및 Wireshark 로고]

■ 후킹

- 앱의 런타임 동작 흐름 파악을 통해 앱 분석 시 필요
- 후킹을 통해 함수 로직 변경, 반환 값 변경 등을 수행하여 동적 바이너리 계측 수행
 - 암호화되기 전 데이터, 호출되는 API, 실행중인 프로세스 등 확인 가능
- 대표 도구: Frida



[Frida 로고]

그 외 분석 방법론

■ 모니터링을 통한 분석

■ 로그 모니터링

- 앱 실행 과정에서 생기는 디버그 로그 및 시스템 로그를 통해 앱 동작 분석
 - 오류 메시지, API 호출, 인증 과정 등 확인 가능
- 대표 도구: logcat, bugreport

```
C:\Users\eunchaeKIM>adb logcat | findstr com.android.vending
03-10 01:48:21.838 2793 2793 D PluginAODManager: onNotificationInfoUpdated() 0|com.android.vending|-555892737|null|102
60
```

[logcat 명령어를 통해 Google Play Store AOD (Always On Display) 알림 확인]

 FS	파일 폴더
 lshal-debug	파일 폴더
 proto	파일 폴더
 dumpstate_board.txt	텍스트 문서
 dumpstate_log.txt	텍스트 문서
 dumpstate-2026-03-10-18-44-41.txt	텍스트 문서
 main_entry.txt	텍스트 문서
 version.txt	텍스트 문서
 visible_windows.zip	압축(ZIP) 폴더

[bugreport 명령어를 통해 로그 데이터 추출]

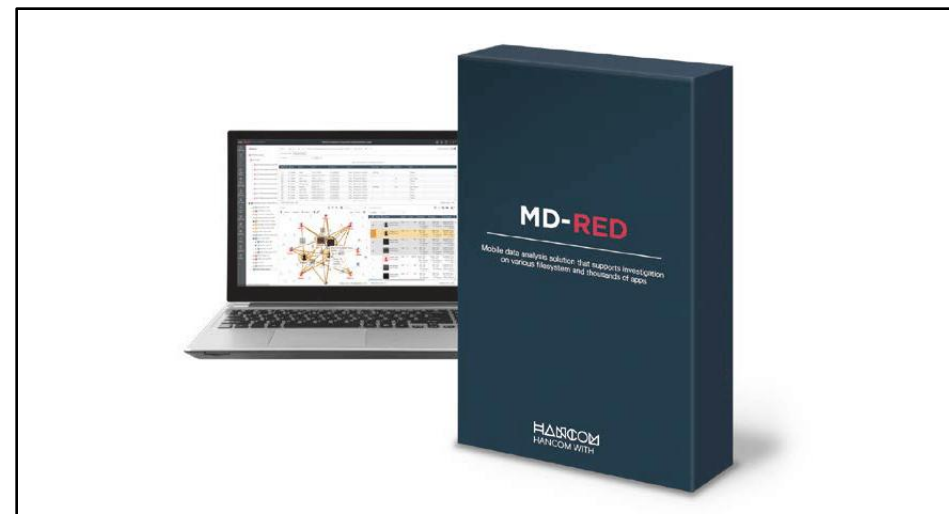
MD-RED

■ 개요

- MD-RED (Mobile Data Recovery and Extraction Device)
- 모바일 장치 데이터 이미지 해독, 복구, 시각화 및 보고 포렌식 소프트웨어

■ 주요 기능

- 모바일 OS, 파일 시스템 및 앱 지원
- 암호화 된 앱 데이터 해독
 - Whatsapp, Facebook 메신저 등
- DB 파일, Hex 등 다양한 타입 데이터 뷰어 지원
- XLS, PDF 등 보고서 생성



[MD-RED]

MD-RED 기본 원리

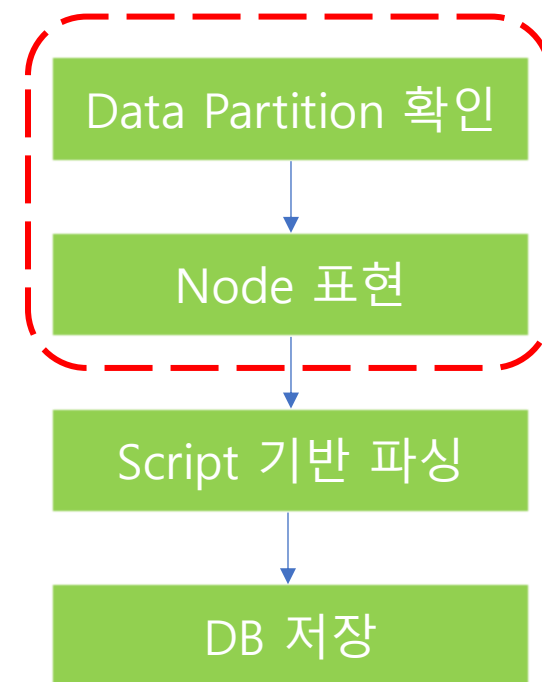
■ Data Analysis 원리 정리

■ Data Partition 확인 (파일 시스템 확인)

- Android 파일 시스템: EXT4, F2FS 등 / iOS 파일 시스템: HFS+, APFS 등
 - 모바일 기기 운영체제 별, 버전 별 파일 시스템 상이
- 파일 시스템, 파티션 테이블, 볼륨 사이즈 정보 등을 먼저 식별

■ Node 표현

- 파일 시스템 식별 후, 각 파일 및 폴더를 공통된 형식으로 표현하는 과정
- Node 구성 요소: State, Name, Size, Extension, Stream 등
- 일정한 형식 → Script 기반 파싱에 용이



MD-RED 기본 원리

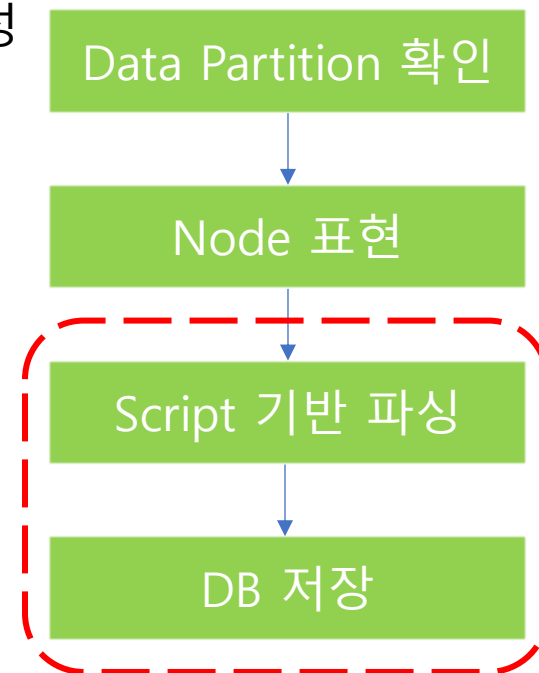
■ Data Analysis 원리 정리

■ Script 기반 파싱

- Script를 통해 노드 탐색 및 데이터베이스 파일 접근 → 파싱 후 Artifact 생성
- Script 종류
 - 내부 정의 Script: MD-RED 내부에서 사용하는 사전 정의 Script (공개 X)
 - 사용자 정의 Script: 사용자가 정의한 Script를 기반으로 파싱 가능
 - 네이밍 규칙: Appname_OS.mdscr*
 - 기본 템플릿 제공 → 템플릿을 기반으로 사용자가 Script 제작 가능

■ DB 저장

- 추출된 아티팩트는 DB에 저장 → MD-RED Explorer에서 분석 결과 확인 가능



* mdscr: MD-RED의 분석 Script 파일 형식으로 Python과 동일 문법 사용

MD-RED 기본 원리

■ 파일 업로드

- MD-NEXT에서 추출한 mdf 파일 업로드 → Extraction Info 확인 가능
 - Extraction Method: logical
 - Hash Info: 추출 당시 SHA256과 비교해 무결성 검증
 - TimeZone: UTC+09:00

Extraction Info

SM-N971N

Change extraction info

SAMSUNG

Extraction Method: Logical

File Size: 953MB

File Name: me.mdf

Extraction Date/Time: 2026-03-10 02:50:48 ~ 2026-03-10 03:20:07

Hash Info:

[SHA256] D5155CA4E7DE125B17A2BC9BA2D24E8431F904BCDCFF4875E7AED14766A303C5 (Verified: [SHA256] D5155CA4E7DE125B17A2BC9BA2D24E8431F904BCDCFF4875E7AED14766A303C5)

Copy

Verify

TimeZone:

(UTC+09:00) 서울

▼

Analysis Script

COMMON#Android.json

Change

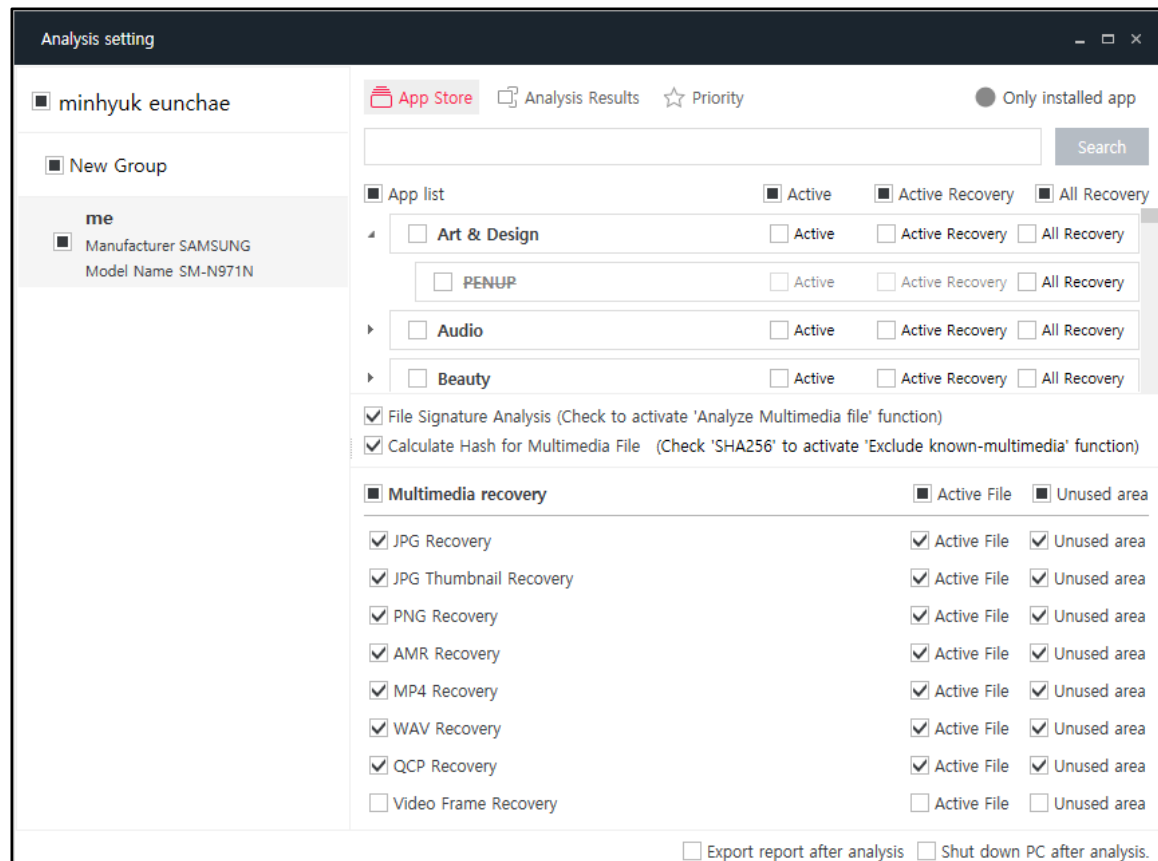
[MD-RED / Extraction Info]

MD-RED 기본 원리

■ Analyze

■ 데이터 선택 후 분석 옵션 선택

- App list: 설치된 앱 목록
 - 분석이 필요한 앱 / 제외할 앱 선택을 통해 분석 시간 단축 가능
- Multimedia recovery: 사진, 영상 등 복구
 - Active File: 현존하는 파일도 복구 범위 포함
 - Unused File: 비할당 영역에서 파일 복구
- Default 설정 값으로 분석 진행



[MD-RED / Analysis Setting]

MD-RED 기본 원리

■ Analyze 결과

- Explorer 탭 내 Analysis Result by App 카테고리 형성
 - Account: 파일 내 사용자 계정 정보 분석
 - Contacts: 연락처 정보 분석
 - Browser History: 북마크, 방문 웹사이트 및 브라우저 정보

Index	App	State	Domain	Item1	Contents1
1	Google Drive	Active	Google Drive	Account	Email : pjh1313123@gmail.com
2	Google Play Movies & TV	Active	Google Play Movies & TV	Account	Email : pjh1313123@gmail.com
3	Schedule Setting (Live)	Active	Schedule Setting (Live)	Account	Email : pjh1313123@gmail.com

Index	App	State	Name	Phone Number	Email
1	Contacts (Live)	Active	Name : Test	Cell : 01011111111	Home : 1111@1111.com
2	Contacts (Live)	Active	Name : Test2	Cell : 01022222222	Home : 2222@2222.com

Analysis Results by App

- ☒ Account (3/3)
- ☒ Information (4/4)
- ☒ Contacts (2/2)
- ☒ Contacts Group (6/6)
- ☒ Browser History (9/9)
- ☒ Log (3/3)
 - ☒ Bluetooth Log (1/1)
 - ☒ Network Log (2/2)
- ☒ Etc (9/9)
- ☒ Applications (247/247)
- ☒ Multimedia (197/342)
 - ☒ Picture (55/55)
 - ☒ Movie (4/4)
 - ☒ Sound (1/1)
 - ☒ Document (44/189)
 - ☒ DB (73/73)
 - ☒ Compressed File (20/20)

[MD-RED / Analysis Result by App]

MD-RED 기본 원리

■ Analyze 결과

■ Analysis Result by App 카테고리 형성

- Bluetooth / Network Log: 연결한 Bluetooth / Network 정보
 - Bluetooth Log: 기기 Name 및 Address 확인
 - Network Log: Wi-Fi SSID 및 비밀번호 추출 확인

Index	App	State	Name	Address
1	Bluetooth Log	Active	BOOK-A1MNPA59SM	D0:57:7E:8A:C5:4A

Index	App	State	Item	Contents
1	Mobile Hotspot (Live)	Active	Mobile Hotspot	ID : #ERROR#SSID#
2	Wifi Profile	Active	Wifi	SSID : DFRC Password : forensicxpert4738

Analysis Results by App

- ✓ Account (3/3)
- ✓ Information (4/4)
- ✓ Contacts (2/2)
- ✓ Contacts Group (6/6)
- ✓ Browser History (9/9)
- ✓ Log (3/3)
 - ✓ Bluetooth Log (1/1)
 - ✓ Network Log (2/2)
- ✓ Etc (9/9)
 - Applications (247/247)
 - Multimedia (197/342)
 - ✓ Picture (55/55)
 - ✓ Movie (4/4)
 - ✓ Sound (1/1)
 - Document (44/189)
 - ✓ DB (73/73)
 - Compressed File (20/20)

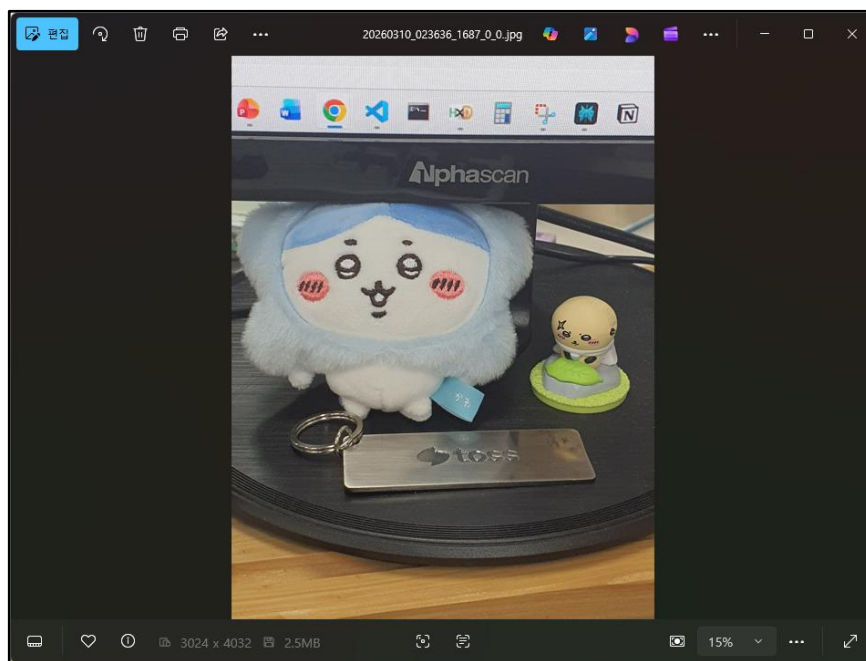
[MD-RED / Analysis Result by App]

MD-RED 기본 원리

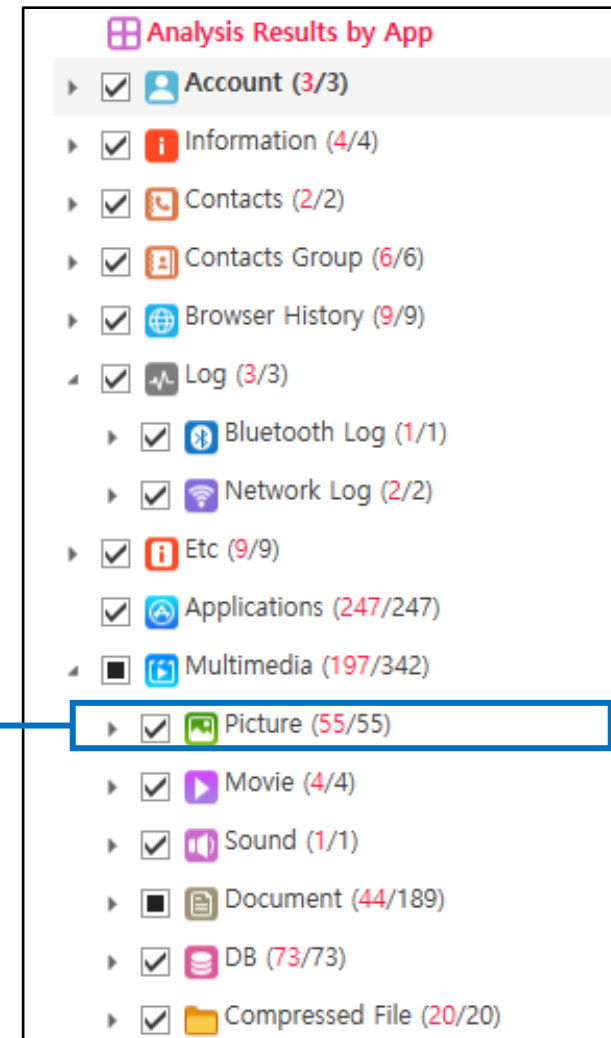
■ Analyze 결과

■ Analysis Result by App 카테고리 형성

- Multimedia: 사진, 영상, 문서 등 추출
 - 사용자가 촬영한 사진 및 캐시 이미지 포함



[Picture 중 일부]



[MD-RED / Analysis Result by App]

MD-RED 기본 원리

■ 보고서 생성

- Create Report → Analyze 자료를 기반으로 PDF 보고서 생성
 - Model: SM-N971N
 - SHA256 → Extraction과 Verified 두 가지 해시 값 제공
 - 데이터 추출 시점 / 데이터 분석 시점 나누어 무결성 검증 가능
- 각 보고서 페이지: 분석 결과 + 파일 별 MD5, SHA1, SHA256 값

State	Attr	Type	Preview	Size Info	Time	App
Active	Recover red	JPG		Width : 4,032 Height : 3,024 File Size : 2,582,413 Offset : 0		Default
File Info	File Path : /media/0/DCIM/Camera File Name : 20260310_023636.jpg					
EXIF	Camera Maker : samsung MetaModel : SM-N971N MetaSoftware : N971NKSU2HWH5 Original Time : 2026-03-10 02:36:37 Digitized Time : 2026-03-10 02:36:37					
Hash	MD5 : 48EB26728D9A7D08E302C7DA884C56C5 SHA1 : AE2F3527594FF46EF7807D63A8833DAC4C0AEA35 SHA256 : BAC5C958739CC3A32D125B6C856B8DBACEDD403AFBD726E3AF05EC6907BD5866					

[MD-RED / 모바일 분석 보고서 내용 일부]

모바일 분석 보고서	
Model	SM-N971N
Manufacturer	SAMSUNG
File Type	Logical
File Size	953MB
File name	me.mdf
Extraction Date/Time	2026-03-10 02:50:48 ~ 2026-03-10 03:20:07
SHA256	Extraction: D5155CA4E7DE125B17A2BC9BA2D24E8431F904BCDCFF4B75E7AED14766A303C5 Verified: D5155CA4E7DE125B17A2BC9BA2D24E8431F904BCDCFF4B75E7AED14766A303C5
Forensic Tool	MD-RED v3.3.11.197

[MD-RED / 모바일 분석 보고서]

도구 실습

ALEAPP

■ 도구 소개

- ALEAPP (Android Logs Events And Protobuf Parser)
- Android 아티팩트 분석 포렌식 도구
- 주요 기능
 - 데이터 소스 파싱 (.tar, .zip, .gz)
 - 다양한 아티팩트 분석 (Gmail, SMS, Device Info 등)
 - HTML 기반 보고서 생성

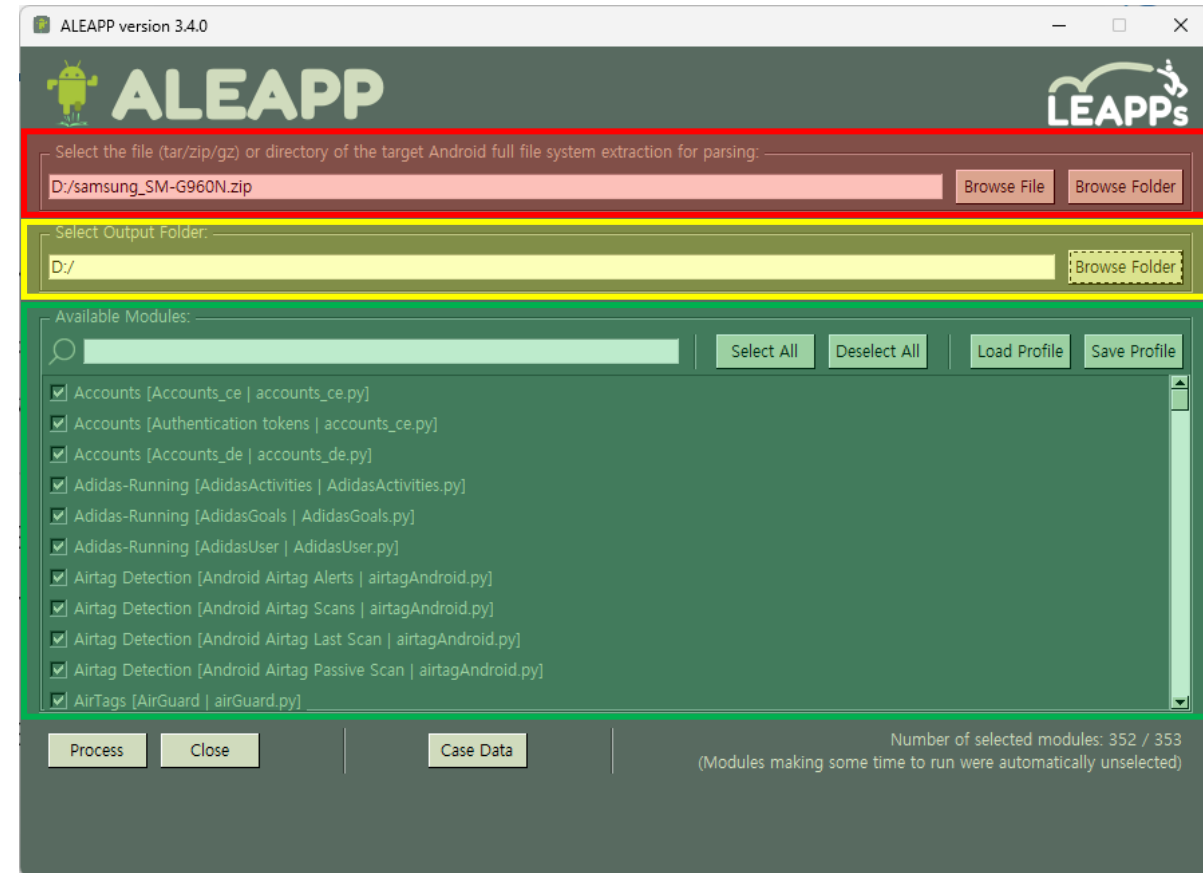


[ALEAPP 로고]

ALEAPP

■ GUI 설명

- Input: tar / zip / gz 선택 가능
- Output 폴더 경로 설정
- Available Modules
 - 353개의 모듈 존재
 - Default: 모두 선택



[ALEAPP GUI]

ALEAPP

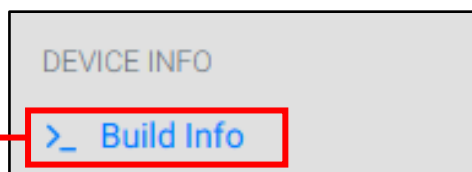
■ 주요 모듈 정리

- Device Info: 기기 식별 정보 파싱 모듈 (build.prop, telephony.db 등)
- Installed App: 설치 앱 목록 및 설치일 파싱 모듈 (packages.list, packages.xml)
- Contacts: 주소록 파싱 모듈 (contacts2.db)
- SMS & MMS: 문자 메시지 기록 파싱 모듈 (mmssms.db)
- Email: Google 계정 및 Gmail 파싱 모듈 (bigTopDataDB.[user_id] 등)

ALEAPP

■ Device Info

- Build Info – build.prop 파싱
 - Key - Value 쌍 구성
 - Android Version / Brand / Device / Model / SDK



Key	Value
Android Version	10
Brand	samsung
Device	starlteks
Model	SM-G960N
SDK	29

[DEVICE INFO – Build Info 확인]

ALEAPP

■ Installed App

- Google Play Links for Apps: Google Play에서 다운받은 App의 Links 리스트 (packages.list)
 - 패키지명, 설치 시간, 업데이트 시간, 설치 경로 확인 가능
- Packages: 현재 설치된 패키지 리스트 (packages.xml)
 - Bundle ID – Possible Google Play Store Link 쌍으로 구성 (Possible: 항상 유효 링크는 X)

INSTALLED APPS

Google Play Links for Apps

Packages

[INSTALLED APPS 확인]

ft	Name	Install Time	Update Time	Install Originator	Installer	Code Path
2008-12-31 15:00:00	com.samsung.android.provider.filterprovider	2008-12-31 15:00:00	2008-12-31 15:00:00			/system/app/FilterProvider
2008-12-31 15:00:00	com.sec.android.app.DataCreate	2008-12-31 15:00:00	2008-12-31 15:00:00			/system/app/AutomationTest_FB

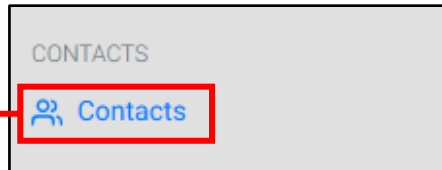
Bundle ID	Possible Google Play Store Link
android	https://play.google.com/store/apps/details?id=android
android.auto_generated_rro_vendor__	https://play.google.com/store/apps/details?id=android.auto_generated_rro_vendor__
android.autoinstalls.config.samsung	https://play.google.com/store/apps/details?id=android.autoinstalls.config.samsung

ALEAPP

■ Contacts

■ contacts2.db 파싱

- Mimetype, Data 1 (전화번호), Display Name, Phone Number, Email Address, Source File
 - Email Address의 경우 연락처에 추가된 경우에만 파싱



Mimetype	Data 1	Display Name	Phone Number	Email Address	Source File
vnd.android.cursor.item/phone_v2	+821062137507	+821062137507	+821062137507		\\?\D:\ALEAPP_Reports_2026-03-10_Tuesday_000516\data\Dump\data\data\com.samsung.android.providers.contacts\databases\contacts2.db
vnd.android.cursor.item/phone_v2	+821062137507	+821062137507	+821062137507		\\?\D:\ALEAPP_Reports_2026-03-10_Tuesday_000516\data\Dump\data\data\com.samsung.android.providers.contacts\databases\contacts2.db
vnd.android.cursor.item/phone_v2	+821041676150	JK	+821041676150		\\?\D:\ALEAPP_Reports_2026-03-10_Tuesday_000516\data\Dump\data\data\com.samsung.android.providers.contacts\databases\contacts2.db
Mimetype	Data 1	Display Name	Phone Number	Email Address	Source File

[CONTACTS - Contacts 확인]

ALEAPP

■ SMS & MMS

- SMS (Short Message Service): 160자 제한 텍스트만 전송 가능
- MMS (Multimedia Message Service): 사진 및 긴 길이의 메시지 전송 가능
- mmssms.db 파싱



[SMS & MMS 확인]

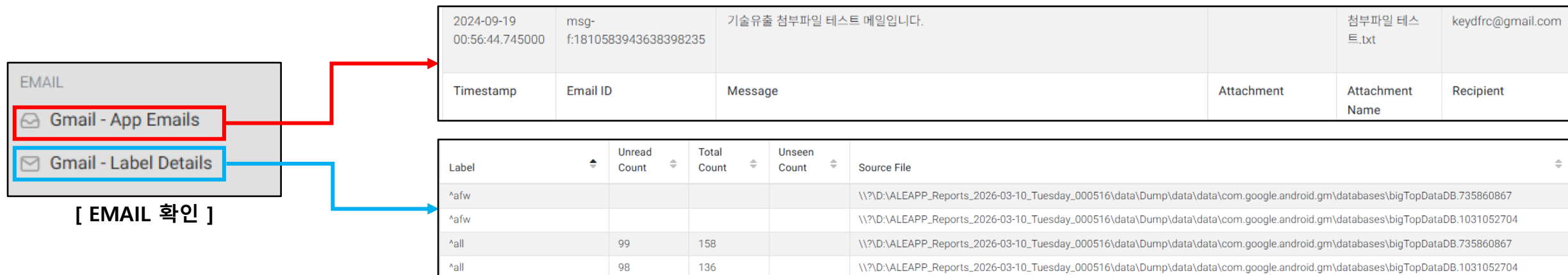
Date	MSG ID	Thread ID	Date sent	Read	From	To	Cc	Bcc	Body
2025-01-07 01:27:35	2	1	2025-01-07 01:27:30	1	01079323685				아닙니다. 오히려 너무 재미있게 들었습니다. 혹시 관련해서 이야기 더 나누고 싶으시면 언제든 연락 주세요. 제가 명함도 같이 보내드립니다.
2025-01-07 01:27:47	4	1	2025-01-07 01:27:44	1	01079323685				 李放 Li Fang 中国北方工业对外合作室主任 중국북방공업 대외협력실장 职责范围 담당업무 - 国际合作与交流 - 국제 협력 및 교류 - 对外战略制定 대외협력실장

Date	MSG ID	Thread ID	Address	Contact ID	Date sent	Read	Type	Body	Service Center	Error code
2025-01-07 01:26:30	1	1	01079323685		2025-01-07 01:26:25	1	Received	안녕하세요, 잘 들어 가셨나요? 어제 이야기는 정말 흥미로웠습니다. 제가 평소에 관심 있던 주제라 더 듣고 싶었어요.	+821029192608	-1
2025-01-07 01:27:14	2	1	01079323685			1	Sent	아... 어제는 제가 좀 취해서 실수했어요. 신경 쓰지 마세요.		0
2025-01-07 01:28:27	3	1	01079323685			1	Sent	... 무슨 제안인지요??		0

ALEAPP

■ Email

- Gmail - App Emails: 실제 사용자가 주고 받은 이메일 내용
 - Timestamp, Email ID, Message, Attachment, Attachment Name, Recipient 등
- Gmail - Label Details: 메일 관리 방식 및 라벨 설정
 - Label, Unread Count, Total Count 등
- bigTopDataDB.[user_id] 파싱



Frida Hooking Basics

■ 도구 소개

- 동적 바이너리 분석 및 주입 오픈소스 도구
- 실행 중인 프로세스에 자바스크립트 주입 → Hooking / 데이터 수정

■ 주요 구성 요소

- Frida Server: 데이터를 추출할 대상에서 실행되는 데몬
- Frida Client: 분석가 PC에서 명령을 내리는 CLI 도구 (스크립트)
- Frida Gum: 실제 인라인 후킹을 담당하는 핵심 엔진 / C 기반

■ 주요 기능

- 함수 Hooking: 특정 함수 호출 시 인자 값을 확인하거나 반환 값 조작
- 앱 동작 모니터링: 실시간 API 호출 추적 및 로그 출력으로 내부 동작 관찰

Frida Hooking Basics

■ 장점

- 실시간 코드 주입 및 동작 확인 가능
- Windows, macOS, Linux, Android, iOS 등 다양한 운영체제와 환경에서 동작
- 루팅 탐지 및 보안 솔루션 우회 가능
- 다양한 API 지원 → API 후킹, 메모리 조작, 함수 호출 수행 가능

■ 단점

- 후킹 스크립트 작성을 위해서 JavaScript와 Frida API 학습 필요
- 후킹 과정에서 대상 앱 불안정성 증가 및 성능 감소
- 시스템 레벨에 접근하기 위해 기기 루팅 또는 탈옥 필요

Frida Hooking Basics

■ 데이터 분석

■ 실습 환경

- 기기 및 버전: Samsung Galaxy Z Flip 3, Android 15

■ 실습 시나리오

- Frida를 통한 '시스템 시간 조작' 탐지하여 조작 전 / 후 시간 정보 출력
- Frida를 통한 기본 알람 앱 후킹 → 조작된 시간 기준 1분 뒤 알람 설정 가능 여부 파악
- 알람 울리는 기준이 진짜 시간인지 조작된 시간 기준인지 확인

■ 실습 결과

- '시스템 시간 조작' 정상적으로 탐지 가능
- 후킹을 통해 알람 설정 가능
- 조작된 시간 기준으로 알람 울리는 것 확인 가능

Frida Hooking Basics

■ 데이터 분석

- 기기의 CPU 아키텍처 조회

```
star21teks:/ # getprop ro.product.cpu.abi  
arm64-v8a
```

[getprop 명령]

- '/data/local/tmp' 경로에 frida-server 설치

```
C:\Users\minhyuk>adb push "C:\Users\minhyuk\frida-server-17.5.2-android-arm64" /data/local/tmp  
C:\Users\minhyuk\frida-server-17.5.2-android-arm64: 1 file pushed, 0 skipped. 28.0 MB/s (52023440 bytes in 1.769s)
```

[adb push 실행을 통한 Frida Server 설치]

- Frida Server 백그라운드 실행

```
star21teks:/data/local/tmp # ./frida-server-17.5.2-android.arm64 &  
[1] 2962
```

[Frida 서버 실행]

Frida Hooking Basics

■ 데이터 분석

■ 시간 조작 탐지 Frida 스크립트 작성

핵심 기능	설명
Java.perform()	Android 앱의 Java 클래스에 접근하기 위한 진입 지점
Java.use()	지정한 Java 클래스를 사용
System.currentTimeMillis()	시스템의 현재 시간을 ms 단위로 변환하는 API 앱에서 시간 값 조회 시 호출
System.currentTimeMillis.implementation()	Frida 후킹을 통해 시간값 확인하고 추가 로직 수행, 콜백 함수 포함
console.log()	Frida 실행 중 탐지 결과나 상태 정보 콘솔에 출력

```

Java.perform(function () {
    const System = Java.use("java.lang.System");
    let lastTime = Number(System.currentTimeMillis());
    const THRESHOLD = 5000; // 5초

    function checkTimeChange(currentTime) {
        currentTime = Number(currentTime);
        let diff = currentTime - lastTime;

        if (Math.abs(diff) > THRESHOLD) {
            console.log("=====");
            console.log("[!] 시스템 시간 변경 감지됨");
            console.log("[+] 조작 전 시간: " + new Date(lastTime).toString());
            console.log("[+] 조작 후 시간: " + new Date(currentTime).toString());
            console.log("[+] 시간 차이(ms): " + diff);

            console.log("=====");
        }

        lastTime = currentTime;
    }

    System.currentTimeMillis.implementation = function () {
        let currentTime = this.currentTimeMillis();
        checkTimeChange(currentTime);
        return currentTime;
    };

    console.log("[*] 시간 조작 탐지 스크립트 탐지");
});
    
```

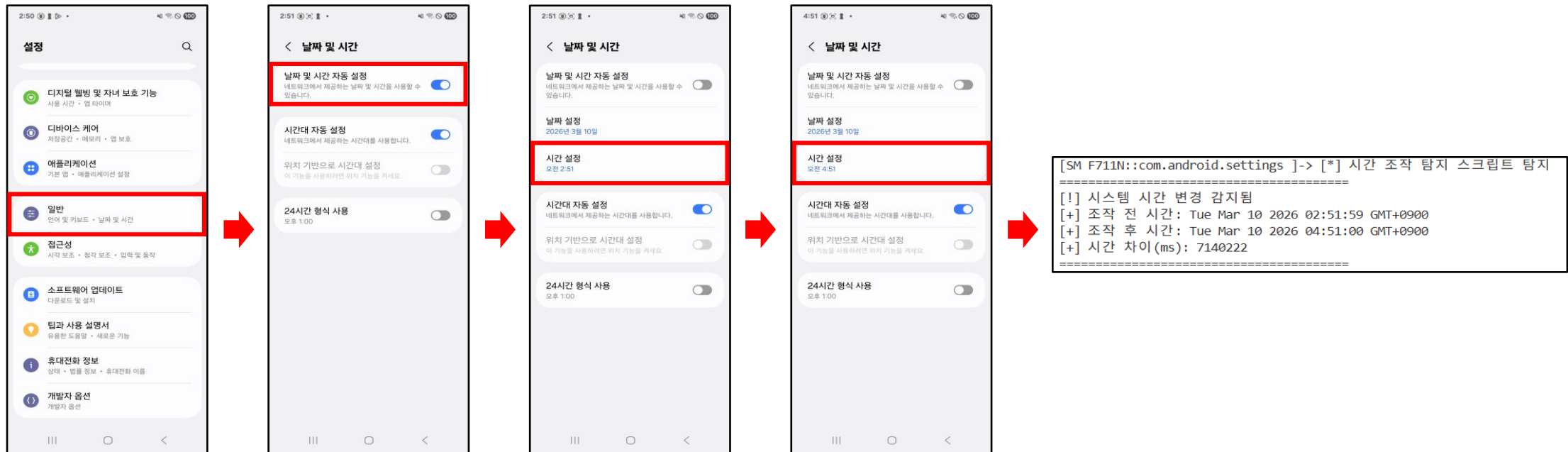
[시간 조작 탐지 Frida 스크립트]

Frida Hooking Basics

■ 데이터 분석

■ 시스템 시간 조작 및 결과 확인

- '설정 → 일반 → 날짜 및 시간 → 날짜 및 시간 자동 설정 비활성화'
- 2026-03-10T02:51 → 2026-03-10T04:51 (현재 시간 기준 2시간 뒤로 시간 조작)



[시스템 시간 조작 과정 및 결과 확인]

Frida Hooking Basics

■ 데이터 분석

■ 알람 후킹 Frida 스크립트 작성

- Intent: 앱의 컴포넌트 사이에서 작업을 요청하고 데이터를 전달하는 객체
- Flag '0x10000000': FLAG_ACTIVITY_NEW_TASK, 새로운 Task에서 Activity 생성 Flag

핵심 기능	설명
Intent.getIntExtra()	Intent에 전달된 데이터 중 정수형 값을 가져오는 메소드
Java.scheduleOnMainThread()	UI 관련 작업 시 메인 스레드에서 코드 실행하도록 예약
Java.choose()	특정 클래스의 객체 선택
intent.addFlags()	Intent 실행 시 동작 방식을 제어하는 플래그 추가
context.startActivity()	지정한 Intent 기반으로 새로운 Activity 실행하는 메소드

```
Java.perform(function () {
    var Intent = Java.use("android.content.Intent");
    var ActivityThread = Java.use("android.app.ActivityThread");
    var Calendar = Java.use("java.util.Calendar");

    console.log("[*] Android 15 - Samsung 알람");

    Intent.getIntExtra.implementation = function (name, defaultValue) {
```

--- 중략 ---

```
var intent = Intent.$new("android.intent.action.SET_ALARM");
intent.setPackage("com.sec.android.app.clockpackage");
intent.putExtra("android.intent.extra.alarm.HOUR", now.get(11));
intent.putExtra("android.intent.extra.alarm.MINUTES", now.get(12));
intent.putExtra("android.intent.extra.alarm.SKIP_UI", true);
intent.addFlags(0x10000000);

context.startActivity(intent);
console.log("[+] 알람 설정 요청 완료 (" + now.get(11) + ":" + now.get(12) + ")");
```

[알람 앱 후킹 Frida 스크립트]

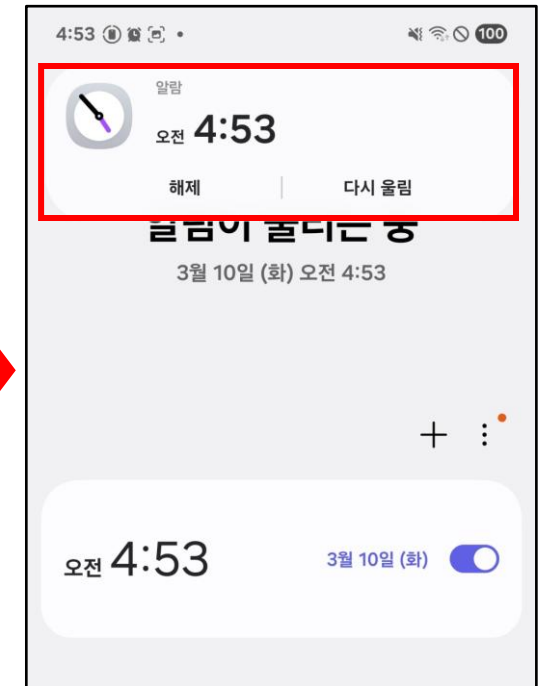
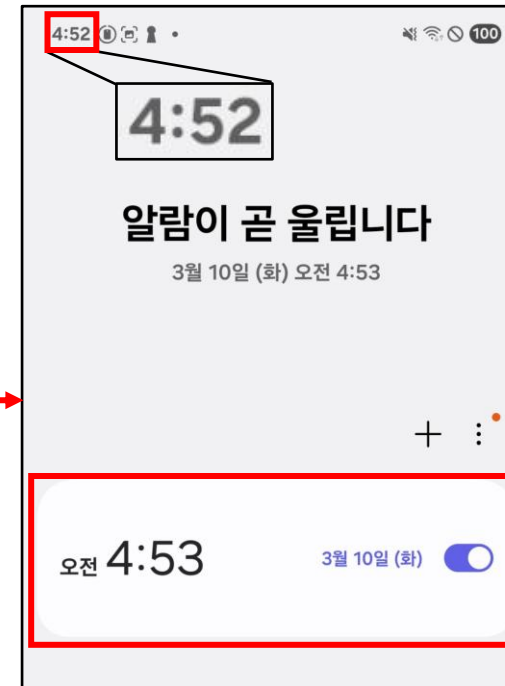
Frida Hooking Basics

■ 데이터 분석

- 알람 앱 후킹을 통한 알람 설정 및 결과 확인
 - 2026-03-10T04:53으로 알람 설정 (조작된 시간 1분 뒤)
 - 조작된 시간 기준으로 알람이 울리는 것 확인 가능

```
[SM F711N::com.sec.android.app.clockpackage ]-> [*] Android 15 - Samsung 알람  
[+] 알람 설정 요청 완료 (4:53)
```

로그 출력 → 기기 내 알람 앱
확인



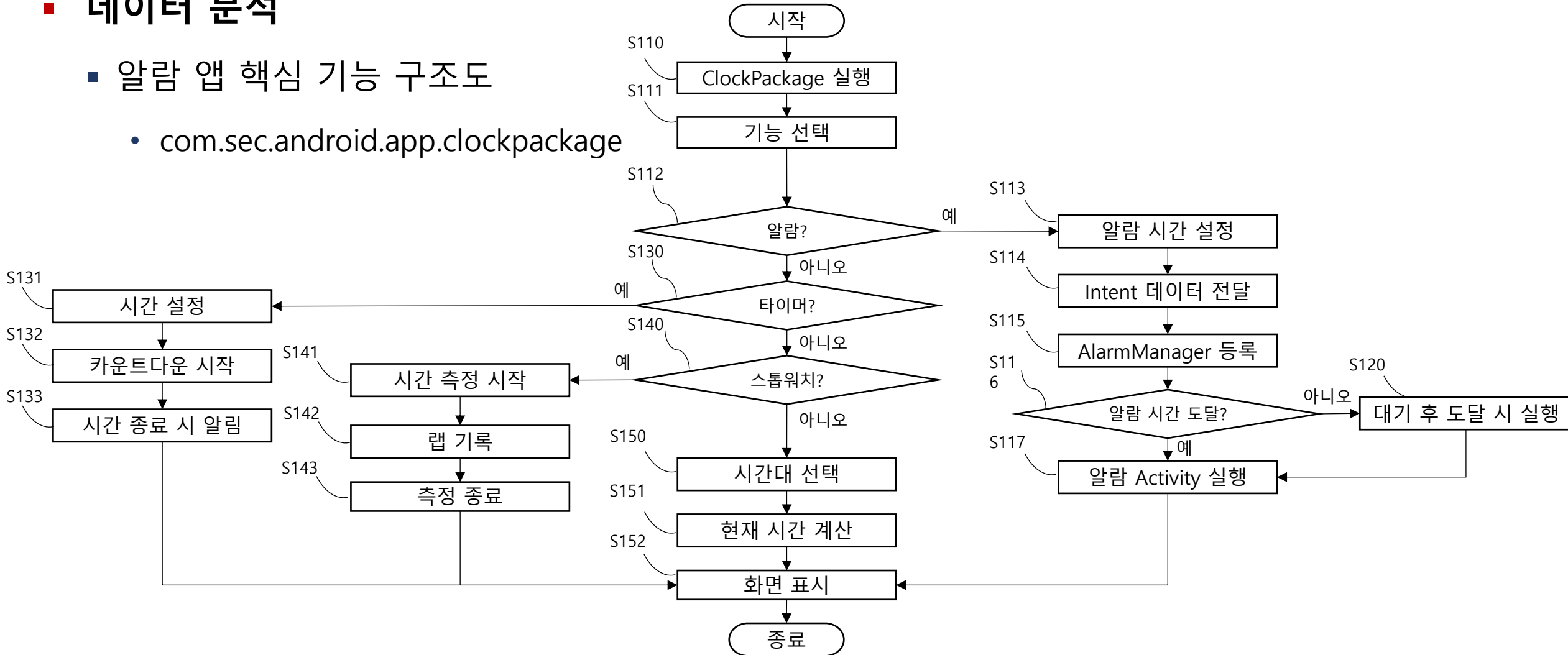
[알람 앱 후킹 및 결과 확인]

Frida Hooking Basics

■ 데이터 분석

■ 알람 앱 핵심 기능 구조도

- com.sec.android.app.clockpackage



THANK YOU
for Listening!



Digital Forensic Research Center

고 려 대 학 교 정 보 보 호 연 구 원
디 지 털 포 렌 식 연 구 센 터

dfrc.korea.ac.kr

Questions?

